



SCHOOL OF ENGINEERING

Implement an incremental development approach in MLOps for a computer vision pipeline.

Supervisor: Brahim El Boudani

Contents

1	Introduction	7
1.1	Dissertation Overview	7
1.2	Aims	7
1.3	Objectives	7
1.4	Project Management	8
1.5	Risk Assessment	8
2	Literature Review	9
2.1	Introduction to MLOps	9
2.1.1	Defination Of MLOps	9
2.1.2	Historical Context: DevOps and CI/CD as Foundation Of MLOps	10
2.1.3	Importance of MLOps for Deploying and Managing ML Models in Production Environments	12
2.2	Scope of MLOps	13
2.2.1	Essential Components of MLOps Systems	13
2.2.2	Scope and Application Areas of MLOps	14
2.3	MLOps Lifecycle and Workflow	15
2.3.1	Essential Phases Of MLOps	15
2.3.2	Iterative Nature and Interdependencies of ML Phases	16
2.3.3	Importance Of Continuous Integration(CI), Continuous Delivery(CD), Continuous Training(CT)	16
2.3.4	Scalability, Governance, and Compliance in MLOps	17
2.4	Tools and Technologies	17
2.4.1	Containerization and Orchestration	17
2.4.2	Version Control	18
2.4.3	Modeling Frameworks	18
2.4.4	Deployment Tools	18
2.4.5	CI/CD Tools	19
2.4.6	Workflow Orchestration	19
2.4.7	Pipeline Automation and Deployment Tools	20
2.5	Challenges In MLOps, CRISP-DM and SEMMA	20
2.5.1	Challenges Associated with CRISP-DM and SEMMA Methodologies	20
2.5.2	Challenges In MLOps	21
2.6	Frameworks and Maturity Models in MLOps	23
2.6.1	MLOps Frameworks	23
2.6.2	MLOps Maturity Models	24
2.6.3	Case Studies Highlighting MLOps Maturity	25
2.7	Comparative Analysis of MLOps Approaches	25
2.7.1	Review and Comparative Analysis of case studies and Pararical Applications of MLOps	25
2.7.2	Categorization of MLOps Practices and Suitability for Various Project Types	26
2.7.3	Insights From Paratical Implementation	26
2.7.4	Specific Considerations For Computer Vision Projects	27
2.8	Sustainability and Future Directions For MLOps	27

2.8.1	Sustainable Practices in MLOps	27
2.8.2	Emerging Trends and Challenges	27
2.8.3	Research Gap and Future Opportunities	28
2.9	Summary and Conclusion	28
3	Methodology and Analysis Procedure	30
3.1	Overview Methodology	30
3.2	Data preprocessing and Model Training	30
3.3	Application Development With Streamlit	31
3.4	Version Control and Collaboration via GitHub	31
3.5	Containerization and Deployment Automation with Docker and GitHub Actions	31
3.6	Kubernetes Deployment and Argo CD	32
3.7	Analysis and Optimization	32
4	Project Development	32
4.1	Data Acquisition	33
4.2	Initial Model Training with CNN	33
4.3	Transformer-Based Model Implementation	34
4.4	Streamlit App Development	36
4.5	Version Control and GitHub Integration	38
4.6	Dockerization	38
4.7	Kubernetes Deployment	40
4.8	CI/CD with GitHub Actions and ArgoCD	42
5	Conclusion	44
6	Personal Reflection	45
A	CNN Code	48
B	Transformers Code	56
C	App Code	62
D	Dockerfile Code	65
E	GitHub Actions Code	65
F	apllication.yaml(ArgoCD) Code	67

List of Figures

1	Gant Chart	8
2	Overview Of MLOps (Symeonidis et al. 2022).	12
3	Trends for MLOps searches online for the year 2020 alone; a variant super-linear growth is evident (Tamburri 2020).	13
4	MLOps Life-cycle, Source: (Symeonidis et al. 2022).	16
5	MLOps Framework, Source:(John, Olsson, and Bosch 2021)	24
6	MLOps in Industry 4.0 Overview, Source: (Faubel and Schmid 2024) . .	25
7	Proposed Methodology Diagram	30
8	Project Workflow	33
9	Streamlit App	37
10	GitHub Repo	38
11	Docker Hub Image	39
12	Kubernetes Deployment and Service	41
13	GitHub Actions	43
14	ArgoCD Integration	44

List of Tables

1	Risk Assessment	9
2	Execution Time Comparison	14
3	Key Components of the MLOps Pipeline, Source: (John, Olsson, and Bosch 2021)	17
4	MLOps Tools by Category	20
5	MLOps Maturity Levels	24
6	Comparison Framework between the MLOps and AIOps Paradigms (Source: (Diaz-De-Arcaya et al. 2023))	26
7	Research Opportunities in MLOps Gap Areas	28

Acknowledgement

First and foremost, I would like to express my sincere thanks to London South Bank University for providing the academic environment, resources, and continuous support that enabled me to undertake and complete this project. The university's focus on critical thinking, innovation, and real-world application has laid a solid foundation for this work. I am deeply grateful to my academic supervisor, **Mr. Brahim El Boudani**, whose guidance, motivation, and insightful feedback were instrumental in shaping the quality and direction of this project. His expertise in machine learning, deep learning, and MLOps significantly influenced the development of this work.

I also wish to extend my appreciation to the faculty and support staff at the School of Engineering and the Department of Computer Science and Informatics. Their academic input and administrative support played a vital role in the smooth progression of my research.

A heartfelt thank you goes to my fellow students and research colleagues at LSBU. Their collaboration, stimulating discussions, and shared passion for AI and data science enriched my learning experience and inspired new perspectives. Lastly, I am profoundly thankful to my family and friends. Their unwavering support, patience, and belief in my abilities have carried me through the most challenging phases of this journey. Their encouragement has been a constant source of strength.

This work would not have been possible without the collective support of all these individuals who contributed to my academic and personal growth. For that, I remain truly grateful.

Student Name: Ali Raza

Student ID: 4302292

Abstract

Machine Learning Operations (MLOps) has emerged as a pivotal discipline that bridges the gap between data science and software engineering, enabling scalable, reproducible, and maintainable deployment of machine learning models in production environments. This dissertation investigates the role of MLOps in contemporary AI workflows, focusing on its application within a real-world computer vision system for violence detection in video streams.

The study begins with a comprehensive literature review, tracing the evolution of MLOps from traditional frameworks like CRISP-DM and SEMMA to modern, DevOps-inspired methodologies. It outlines the essential principles of MLOps—Continuous Integration (CI), Continuous Delivery (CD), and Continuous Training (CT)—and identifies their practical significance in automating the machine learning lifecycle. Furthermore, it explores the scope of MLOps across industries, highlights key challenges such as model versioning, reproducibility, collaboration barriers, and operational scalability, and critically assesses current tools and frameworks like Docker, Kubernetes, ArgoCD, GitHub Actions, and Streamlit.

A practical project is executed to demonstrate the incremental development and deployment of a violence detection system using both CNN and Transformer-based models. The project integrates version control, containerization, and CI/CD practices into a modular and reproducible pipeline. GitHub was used for collaborative development, Docker and Kubernetes for deployment automation, and ArgoCD for synchronization with Git repositories. This hands-on implementation underscores the iterative nature of MLOps, emphasizing the importance of continuous monitoring and feedback loops in maintaining model performance and operational robustness.

The dissertation concludes by reflecting on the sustainability and future directions of MLOps, advocating for responsible AI practices and further research into interoperability, automation, and governance. Through both theoretical analysis and practical application, this work contributes to the growing body of knowledge in MLOps and provides a blueprint for organizations seeking to operationalize machine learning in complex and dynamic environments.

1 Introduction

1.1 Dissertation Overview

Machine Learning Operations (MLOps) represents a critical evolution in deploying and managing machine learning (ML) models within production environments, effectively bridging the gap between traditional data science methodologies and modern operational requirements.

The adoption of machine learning has exponentially increased across various industries, highlighting the necessity for structured and efficient methodologies that ensure seamless integration, deployment, and maintenance of ML solutions.

This dissertation presents a comprehensive analysis of MLOps, including its foundational principles derived from DevOps, the essential components, lifecycle, scope of application, challenges, and the contemporary tools and frameworks supporting its implementation. By examining the evolution from traditional methodologies such as CRISP-DM and SEMMA to the more robust and iterative MLOps framework, the research underscores the critical need for an automated, collaborative, and scalable approach to deploying ML systems.

1.2 Aims

The primary aim of this dissertation is to critically evaluate the existing literature and practical applications of MLOps to provide a detailed comparative analysis, identify current challenges, and propose methodologies to optimize the implementation and management of ML models in production. By synthesizing theoretical insights and practical experiences, the research aims to deliver an integrated view of MLOps, contributing to the body of knowledge by highlighting its significance, effectiveness, and areas requiring further research and development.

1.3 Objectives

To achieve the overarching aims, the following specific objectives have been set:

- To define and conceptualize MLOps clearly, establishing its distinction from traditional ML methodologies and highlighting its key principles derived from DevOps practices.
- To analyze the historical evolution from traditional data science frameworks such as CRISP-DM and SEMMA to the contemporary practices embedded within MLOps.
- To systematically identify and assess the essential components of MLOps, including CI/CD, continuous monitoring, governance, compliance, and scalability within ML operations.
- To critically evaluate existing MLOps frameworks and maturity models, offering insights into their practical applications and effectiveness in various industry contexts, particularly focusing on complex domains such as computer vision.
- To identify key challenges associated with the adoption and implementation of MLOps, addressing both technical and organizational barriers, as well as cultural resistance within organizations.

- To explore the tools and technologies integral to successful MLOps implementations, including containerization, version control, deployment tools, and pipeline automation platforms.
- To execute an applied project that demonstrates the practical application of MLOps principles by training a model, developing an application, managing code with GitHub, containerizing using Docker, deploying via Kubernetes, and using ArgoCD for synchronization.
- To examine the sustainability and future directions of MLOps, identifying emerging trends, research gaps, and opportunities to advance the field further.

1.4 Project Management

Project management was pivotal in steering the successful implementation of this dissertation titled "Implement an Incremental Development Approach in MLOps for a Computer Vision Pipeline." Given the interdisciplinary nature of the work—spanning data science, machine learning, DevOps, and software engineering—a structured, phased, and incremental approach was essential to ensure quality, traceability, adaptability, and timely delivery.

The core objective of this project was to apply MLOps practices to develop and operationalize a machine learning model for violence detection in videos using both CNN and Transformer-based architectures. The development cycle was organized according to agile principles and an incremental delivery approach. This method enabled iterative prototyping, continuous integration of new findings, and responsiveness to unforeseen challenges.

Here is the gantt chart for project management of this project:

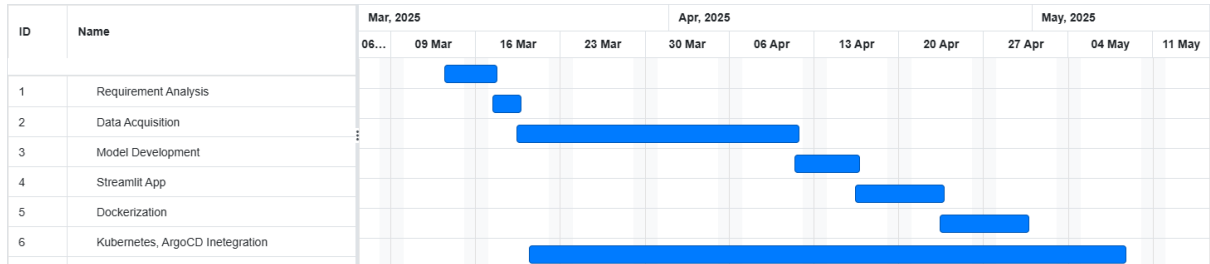


Figure 1: Gantt Chart

This Gantt chart presents a comprehensive overview of the project timeline spanning from 13 March 2025 to 9 May 2025, structured across eight weeks.

1.5 Risk Assessment

A proactive risk management strategy was implemented to address potential technical and scheduling challenges:

Table 1: Risk Assessment

Risk	Probability	Impact	Mitigation Strategy
Incomplete dataset or corrupted video files	Medium	High	Validated files via SHA-checksums; maintained backup
Poor CNN performance	High	Medium	Pivoted to Transformer-based models (TimeSformer)
Streamlit compatibility issues	Medium	Low	Used containerization to isolate dependencies
GitHub Actions runtime failures	Low	High	Employed caching and container-layer validation
Kubernetes deployment conflicts	Medium	High	Validated manifests with <code>kubect1 dry-run</code> and used <code>minikube</code> locally for simulations
ArgoCD synchronization lags	Low	Medium	Enabled auto-heal and manual sync override as fallback

2 Literature Review

2.1 Introduction to MLOps

2.1.1 Defination Of MLOps

Machine Learning Operations, commonly referred to as MLOps, is an interdisciplinary approach that combines machine learning(ML), software engineering, and operations principles to simplify and manage the lifecycle of ML systems in production environments (Kreuzberger, Küh1, and Hirsch1 2023). As defined by (Testi et al. 2022), MLOps refers specifically to "the interactive process of pushing the latest best ML models to production", emphasizing the automation, integration, and continuous monitoring throughout the development, deployment, and maintenance phases of ML applications.

Furthermore, MLOps is characterized as a comprehensive set of practices aimed at managing ML model deployments efficiently and reliably, using automation, continuous integration (CI), continuous delivery (CD) and continuous training (CT) practices (Symeonidis et al. 2022). The MLOps paradigm incorporates the principles of DevOps to ensure a collaborative, systematic, and repeatable process, allowing organizations to consistently deliver high-quality ML-based solutions (Wazir, Kashyap, and Saxena 2023).

2.1.2 Historical Context: DevOps and CI/CD as Foundation Of MLOps

Historically, the evolution of Machine Learning Operations (MLOps) can be understood by examining the development of earlier data science methodologies and frameworks such as CRISP-DM, SEMMA, and the Data Science Life Cycle, leading up to contemporary MLOps practices rooted in DevOps.

Cross Industry Standard Process for Data Mining (CRISP-DM) was developed in the late 1990s as a structured approach to data mining projects. It is characterized by six sequential phases: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment (Chapman et al. 2000). CRISP-DM gained widespread acceptance due to its clear and practical framework for managing data mining projects. However, its linearity and limited feedback mechanisms between deployment and earlier phases are significant drawbacks, particularly when dealing with dynamic and continuously changing datasets in modern ML scenarios (Mariscal, Marban, and Fernandez 2010).

The **SEMMA (Sample, Explore, Modify, Model, Assess)** methodology, developed by SAS Institute, provides another systematic approach explicitly focused on iterative data-driven analytical processes. SEMMA emphasizes exploratory data analysis and iterative refinement of models but has been criticized for its limited focus on the early business-understanding phase and minimal attention toward deployment, monitoring, and ongoing operational management of models post-development (Azevedo and Santos 2008).

Subsequently, the broader concept known as the **Data Science Life Cycle** emerged, incorporating iterative loops and more integrated feedback mechanisms among phases. This lifecycle typically includes phases such as Problem Formulation, Data Acquisition, Data Cleaning, Exploratory Analysis, Feature Engineering, Model Training, Evaluation, Deployment, and Monitoring. The Data Science Life Cycle promotes a more integrated and iterative process, accounting for the need to continuously retrain models as new data arrives and environmental conditions shift ([saltz2018exploring](#)). Despite these enhancements, this life cycle generally lacks the structured, automated, and collaborative infrastructure necessary for managing complex ML models in real-time operational environments.

In contrast to these earlier frameworks, **MLOps**, which evolved directly from DevOps practices, addresses critical limitations inherent in CRISP-DM, SEMMA, and the traditional Data Science Life Cycle. DevOps itself emerged as a methodology designed to bridge the gap between software development and operations through automation, collaboration, and continuous feedback mechanisms, significantly improving software quality and deployment efficiency (Symeonidis et al. 2022). Central to DevOps are Continuous Integration (CI), Continuous Delivery (CD), and Continuous Deployment practices, which systematically automate software delivery pipelines, ensuring rapid and reliable software releases (Zhou, Yu, and Ding 2020).

These foundational DevOps practices have been adapted explicitly to address the distinctive challenges of ML systems, thus giving rise to the specialized discipline of MLOps. Unlike previous methodologies, MLOps explicitly incorporates continuous monitoring, automated retraining (Continuous Training, CT), and robust model governance mechanisms. It emphasizes not only iterative model improvement but also the reliable and scalable deployment, continuous validation, and rigorous operational management of ML models in production environments (Kreuzberger, Kühl, and Hirschl 2023). The neces-

sity of adopting MLOps arises from the increasing complexity, scale, and velocity of data-driven applications and ML model deployments. Traditional frameworks like CRISP-DM, SEMMA, and even the Data Science Life Cycle, though valuable, face significant limitations:

- **Limited Automation and Scalability:** Traditional methodologies such as CRISP-DM, SEMMA, and even the general Data Science Life Cycle were primarily developed during periods when the complexity, volume, and velocity of data were significantly lower. Consequently, they emphasize manual processes and individualized analyses, which are feasible only for small-scale, one-off data projects. These methodologies inherently lack the automated pipeline capabilities and resource orchestration tools necessary to effectively handle the continuous streams of data, frequent retraining cycles, and the large-scale deployment scenarios characteristic of modern ML applications. The absence of built-in automation results in inefficient resource utilization and significantly restricts scalability.
- **Weak Deployment and Operational Management:** Earlier data science methodologies, particularly CRISP-DM and SEMMA, emphasize phases up to model evaluation but pay limited attention to post-deployment management. Deployment is often treated as a final step rather than the beginning of continuous operational management. This approach overlooks the critical necessity of monitoring model performance, detecting model and data drift, and automatically triggering retraining or corrective actions. Consequently, the lack of structured deployment and monitoring processes leads to rapid degradation of model accuracy and reliability in dynamic, real-world environments, ultimately diminishing their effectiveness and utility in long-term operational settings.
- **Insufficient Collaboration and Integration:** Traditional frameworks typically assume isolated workflows handled predominantly by data analysts or statisticians, with minimal integration into broader IT and business operations. This siloed approach prevents effective communication and coordination among different teams—data scientists, software engineers, operational specialists, and business stakeholders. In contrast, modern ML systems require a multidisciplinary approach, with clear roles, shared workflows, and integrated tooling to ensure seamless interaction and exchange of information. The absence of built-in mechanisms to facilitate this collaborative integration leads to misunderstandings, redundancy, inefficiencies, and reduced productivity.
- **Absence of Comprehensive Governance and Compliance:** Traditional frameworks were not designed with rigorous governance, auditing, compliance, and traceability requirements in mind. In highly regulated industries such as healthcare, finance, and automotive, explicit governance structures, thorough documentation, clear traceability of decisions, and reproducibility of outcomes are not merely beneficial—they are mandatory. Without structured mechanisms to manage compliance, auditability, and traceability, earlier methodologies significantly increase the risk of regulatory violations, ethical concerns, and potential legal consequences. This gap becomes increasingly problematic as ML systems become integral to critical and sensitive applications.

These explicit limitations underscore why MLOps, with its emphasis on automation, continuous operational management, collaborative integration, and comprehensive governance, has become an essential evolution in effectively deploying, managing, and sustaining ML systems in production environments.

To clearly visualize this transition and the core elements of the contemporary MLOps lifecycle, Figure 2 provides an overview of MLOps incorporating key principles derived from DevOps along with ML-specific adaptations:

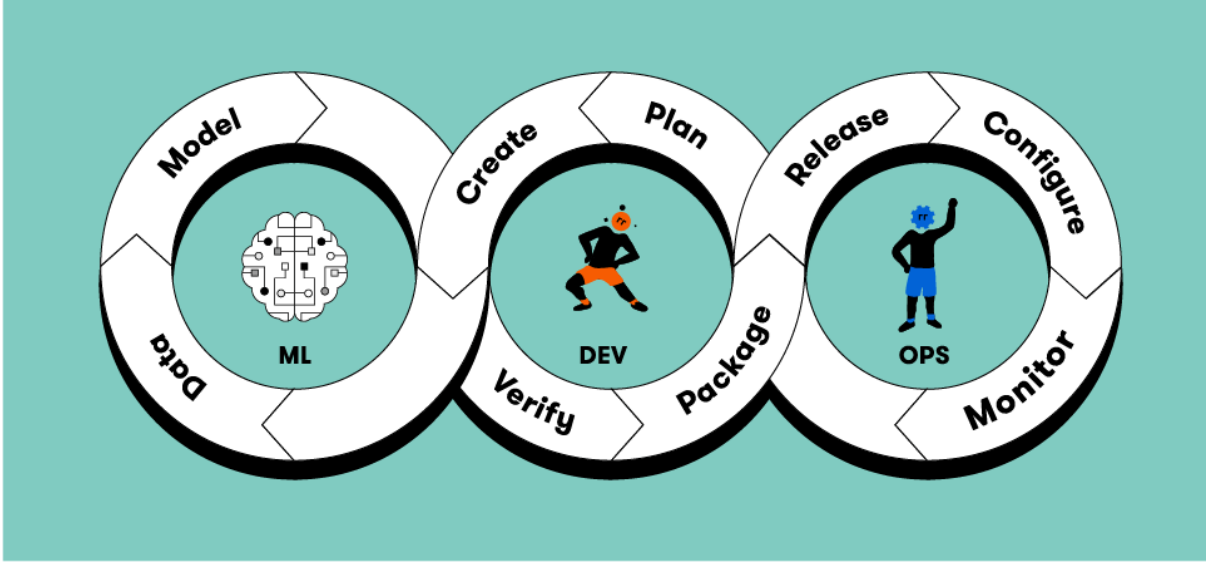


Figure 2: Overview Of MLOps (Symeonidis et al. 2022).

The historical integration of DevOps practices into MLOps highlights the critical advancements made to address the limitations of earlier data science methodologies. By embracing automation, continuous integration, continuous deployment, and continuous training, MLOps has emerged as a robust and comprehensive framework, well-suited to the evolving demands of modern machine learning systems and operations.

2.1.3 Importance of MLOps for Deploying and Managing ML Models in Production Environments

The deployment of machine learning models in production environments presents distinct challenges compared to traditional software systems. Unlike standard software, ML systems continuously evolve due to new data inputs and environmental changes, necessitating frequent retraining and redeployment.

Consequently, MLOps plays a pivotal role in bridging the gap between data science and production environments by providing structured methodologies and technological support to automate and streamline these processes (Bayram and Ahmed 2025).

A robust MLOps infrastructure enhances model performance and significantly reduces the time-to-market, minimizes human error, and ensures consistent quality control across the entire ML lifecycle. According to (Singla 2023), MLOps helps overcome several critical challenges such as model versioning, reproducibility, scalability, and robustness. By facilitating these aspects, MLOps enables businesses to derive sustainable and long-term value from their ML investments, ensuring reliability, scalability, and continuous improvement in the real-world deployment of machine learning applications.

The increasing adoption trends in both academic research and industry practices further highlight the importance of MLOps. Figure 1 illustrates the rapid growth and increasing popularity of MLOps in recent years, as observed from online search trends:

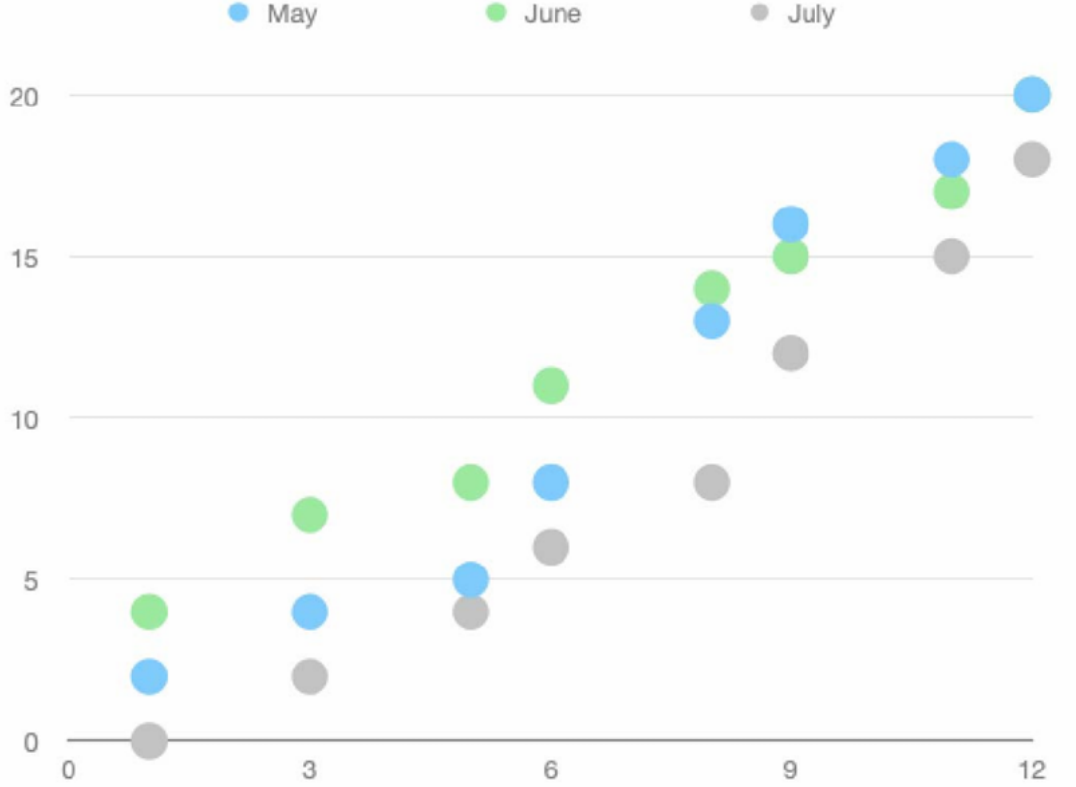


Figure 3: Trends for MLOps searches online for the year 2020 alone; a variant super-linear growth is evident (Tamburri 2020).

The figure clearly demonstrates that the interest in and the need for structured and automated ML lifecycle management through MLOps is continually expanding.

2.2 Scope of MLOps

2.2.1 Essential Components of MLOps Systems

A key distinguishing feature of MLOps is its emphasis on specialized components that facilitate continuous improvement and management of the model. (Warnett and Zdun 2024) emphasize that a comprehensive MLOps system must incorporate robust components for model experimentation, training orchestration, deployment automation, monitoring and alerting mechanisms, and robust version control for data and models. The integration of these components enables organizations to maintain high-quality standards, reproducibility, and traceability of ML models deployed in production environments.

(Zhou, Yu, and Ding 2020) further highlight specific CI/CD pipeline tasks integrated within MLOps to automate ML lifecycle activities effectively. These tasks typically involve data validation, preprocessing, automated model training, and monitoring. Their

study provides quantitative insight into resource and time consumption across these tasks, illustrating the complexity involved (see Table 1).

Table 2: Execution Time Comparison

Model	CI/CD		Kubeflow		Total
	(min)	(%)	(min)	(%)	(min)
DenseNet-121	0.586	0.148	395.933	99.852	396.519
GoogLeNet	0.505	0.105	480.950	99.895	481.455
Inception V3	0.581	0.059	981.083	99.941	981.664
MobileNet	0.600	1.093	54.317	98.907	54.917
ResNet-18	0.857	0.679	125.450	99.321	126.307
ShuffleNet	0.603	0.547	109.633	99.453	110.236
VGG-16	0.460	0.245	187.667	99.755	188.127

2.2.2 Scope and Application Areas of MLOps

The scope of MLOps extends significantly across various industries, driven by the growing dependence on machine learning (ML) models for critical operations, decision-making processes, and customer-facing applications. By integrating best practices from DevOps into the ML lifecycle, MLOps has become essential in ensuring not only the smooth deployment of models but also their continuous monitoring, maintenance, and updating in dynamic environments.

According to (Symeonidis et al. 2022), industries such as finance, healthcare, automotive, manufacturing, and retail are leading adopters of MLOps practices. Each sector utilizes MLOps differently, but the common objectives remain: improving model reliability, enabling scalability, ensuring traceability, and maintaining regulatory compliance.

In the finance sector, MLOps plays a vital role in fraud detection, algorithmic trading, credit risk assessment, and customer churn prediction. Financial institutions leverage continuous model retraining and monitoring to adapt rapidly to market changes and evolving fraud patterns (Symeonidis et al. 2022).

In healthcare, MLOps supports the deployment of predictive models for disease diagnosis, patient monitoring, and personalized treatment planning. Healthcare regulations such as HIPAA demand strict compliance, and MLOps ensures that ML models used in healthcare are explainable, reproducible, and auditable, thus satisfying regulatory requirements (Warnett and Zdun 2024).

In the automotive industry, MLOps facilitates applications in autonomous driving systems, driver-assistance systems, and predictive maintenance. Given the safety-critical nature of these systems, continuous validation and monitoring of deployed models are imperative. The ability of MLOps to enable continuous feedback loops from live systems back to model improvement pipelines is crucial in ensuring vehicle safety and performance (Zhou, Yu, and Ding 2020).

In retail and e-commerce, MLOps supports recommendation systems, dynamic pricing, inventory management, and customer segmentation. Retailers rely heavily on near-real-time model updates based on customer behavior and purchasing patterns, tasks made possible through well-orchestrated MLOps pipelines (Symeonidis et al. 2022).

Furthermore, MLOps is pivotal in Computer Vision (CV) and Video Analysis applica-

tions, domains where the complexity and volume of data significantly exceed those in traditional tabular ML tasks. In Computer Vision, MLOps ensures efficient management of models related to object detection, image segmentation, facial recognition, and OCR (Optical Character Recognition). CV models often require continuous retraining as the visual features in data (e.g., backgrounds, lighting, object types) evolve over time. MLOps practices ensure that these models are robust, scalable, and capable of handling frequent updates without service disruptions.

In video processing applications, such as surveillance systems, sports analytics, content moderation, and autonomous navigation, MLOps pipelines manage the challenges of large-scale data ingestion, frame-by-frame annotation, model training, deployment, and real-time inference.

For example:

- In **smart surveillance**, models must continuously adapt to different lighting conditions, camera angles, and environmental changes.
- In **sports video analysis**, real-time object tracking (e.g., ball, players) requires ultra-low latency model inferencing, retrained regularly based on new game data.

These dynamic scenarios necessitate robust pipelines that not only deploy models efficiently but also monitor data drift and trigger retraining mechanisms as required.

(Zhou, Yu, and Ding 2020) demonstrated how ML pipelines like Kubeflow-based systems handle large computational models effectively, citing their experimental results across several deep learning architectures such as ResNet, MobileNet, and Inception models.

2.3 MLOps Lifecycle and Workflow

2.3.1 Essential Phases Of MLOps

Machine Learning Operations (MLOps) involves several interconnected phases, which collectively facilitate the smooth transition of machine learning (ML) models from conceptualization to production and continuous improvement. According to recent studies, the essential phases of the MLOps lifecycle typically include: Problem Definition, Data Acquisition, Data Preparation, Model Development, Deployment, Monitoring, Management, Scalability, Governance, and Compliance (Symeonidis et al. 2022).

(Symeonidis et al. 2022) clearly present the MLOps lifecycle in their research as a structured cycle involving continuous iterations and feedback loops between phases. The cyclic and iterative nature of MLOps ensures that ML models remain accurate, relevant, and reliable through continuous evaluation and retraining.

Figure 4 clearly illustrates these interconnected phases of the MLOps lifecycle:

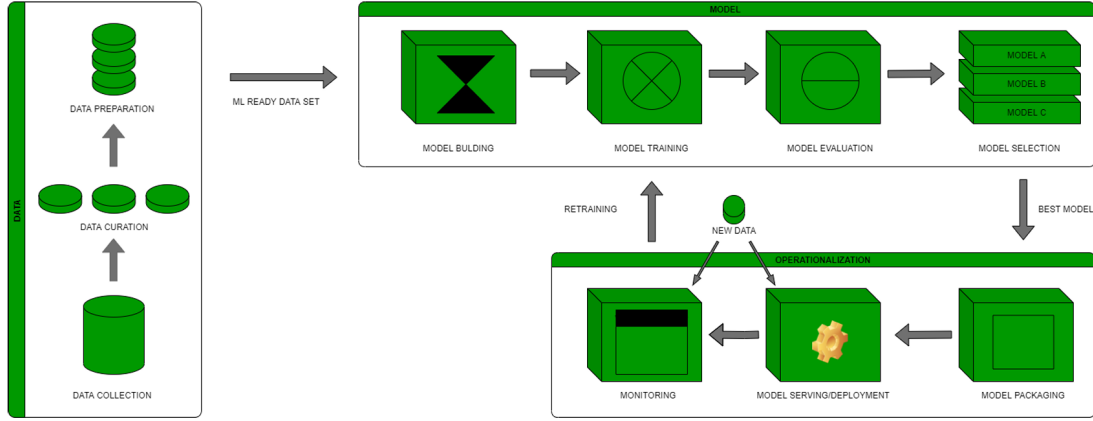


Figure 4: MLOps Life-cycle, Source: (Symeonidis et al. 2022).

The figure effectively highlights that MLOps is not linear but iterative, emphasizing the importance of constant feedback and adaptation based on model performance and evolving data. Additionally, Testi et al. (2022) provide an insightful overview that categorizes the workflow of MLOps clearly into specific phases, offering clarity about the roles and responsibilities at each stage.

2.3.2 Iterative Nature and Interdependencies of ML Phases

A crucial characteristic of MLOps is its inherently iterative and interdependent nature. Unlike traditional software engineering, ML models require continuous refinement to maintain performance in dynamic real-world environments. Each phase of the MLOps lifecycle feeds information back into previous phases. For instance, monitoring can detect changes in data distribution or degradation in model performance, triggering a new cycle of data preparation, model retraining, and redeployment (Symeonidis et al. 2022). According to (Zhou, Yu, and Ding 2020), this iterative approach is critical to ensuring models remain valid and reliable. The monitoring phase is particularly crucial as it provides actionable insights that guide subsequent iterations of model training and improvement. This continuous feedback loop, inherent in MLOps, is essential for maintaining robust and performant ML systems in production.

2.3.3 Importance Of Continuous Integration(CI), Continuous Delivery(CD), Continuous Training(CT)

The principles of Continuous Integration (CI), Continuous Delivery (CD), and Continuous Training (CT) form the backbone of effective MLOps practices. These principles enable systematic and automated management of the ML lifecycle, significantly reducing the complexity and time required to deploy and update ML models in production. Continuous Integration (CI) ensures that all ML code, including data preprocessing scripts and model training algorithms, are regularly merged and tested. This process helps identify integration errors early, improving the reliability of deployments. Continuous Delivery (CD) focuses on automating the deployment of validated ML models into production environments, reducing manual intervention and potential deployment errors. It provides a structured pathway for rapid and safe delivery of incremental model improvements.

Continuous Training (CT) uniquely pertains to MLOps, addressing the dynamic nature of ML applications. CT automatically retrains ML models whenever performance degradation is detected or new relevant data becomes available, ensuring models remain accurate and relevant over time. (John, Olsson, and Bosch 2021) offers a detailed breakdown of how these CI/CD principles specifically apply within the context of MLOps pipelines in their comprehensive study:

Table 3: Key Components of the MLOps Pipeline, Source: (John, Olsson, and Bosch 2021)

CI Components	CD Components	Automated ML Triggering
Data Validation	Model Validation	New Data Availability
Data Preprocessing	Model Deployment	Model Performance Degradation
Model Training	Performance Monitoring	Scheduled Retraining
Model Testing	Automated Rollout	Continuous Feedback

This table effectively summarizes how MLOps practices employ CI/CD automation across different tasks, highlighting the clear roles that CI, CD, and CT play in maintaining a robust ML production environment.

2.3.4 Scalability, Governance, and Compliance in MLOps

Scalability, governance, and compliance are additional key considerations within the MLOps lifecycle.

Scalability ensures that ML systems can effectively manage increasing workloads and data volumes without significant manual intervention. Governance addresses the need for structured management practices around data handling, model deployment, and user access, which are critical for ensuring ethical use of ML systems.

Compliance, especially in regulated industries like healthcare and finance, ensures that deployed models adhere strictly to industry-specific regulatory standards (Testi et al. 2022).

Integration of governance and compliance into the MLOps lifecycle supports transparency, accountability, and trustworthiness in automated decision-making systems. These aspects become increasingly critical as ML systems become more integral to operational decision-making in various industry sectors.

2.4 Tools and Technologies

2.4.1 Containerization and Orchestration

Containerization is pivotal in MLOps for ensuring consistent environments across development, testing, and production.

- **LXC (Linux Containers):** Offers lightweight OS-level virtualization, allowing multiple isolated Linux systems (containers) to run on a single host. It's suitable for scenarios requiring full system containers but lacks the extensive tooling ecosystem found in Docker and Kubernetes.

- **Docker:** Provides a platform for packaging applications and their dependencies into containers, ensuring consistency across environments. Docker’s simplicity and widespread adoption make it a standard choice for containerizing applications.
- **Kubernetes:** An open-source orchestration system for automating deployment, scaling, and management of containerized applications. Kubernetes excels in managing complex, distributed systems and is ideal for large-scale ML deployments.

My Choice: Kubernetes is selected for its robust orchestration capabilities, scalability, and active community support, making it suitable for managing complex ML workflows.

2.4.2 Version Control

Version control systems are essential for tracking changes in code, data, and models, facilitating collaboration and reproducibility.

- **Git:** A distributed version control system that tracks changes in source code during software development. It’s widely used for managing codebases and collaborating among multiple developers.
- **DVC (Data Version Control):** An extension to Git that handles large files, data sets, machine learning models, and metrics, enabling versioning and reproducibility in ML projects.

My Choice: Git is utilized for code versioning, ensuring collaborative development and traceability.

2.4.3 Modeling Frameworks

Selecting the appropriate modeling framework is crucial for developing efficient and scalable ML models.

- **PyTorch:** An open-source ML library developed by Facebook’s AI Research lab, known for its dynamic computation graph and ease of use, making it popular in research settings.
- **Keras:** A high-level neural networks API, written in Python and capable of running on top of TensorFlow, CNTK, or Theano. Keras is user-friendly and suitable for quick prototyping.

My Choice: PyTorch is preferred for its flexibility and dynamic computation graph, facilitating experimentation and model development.

2.4.4 Deployment Tools

Deployment tools are vital for serving ML models and applications to end-users.

- **Flask:** A micro web framework for Python, offering simplicity and fine-grained control, suitable for small-scale applications.
- **Django:** A high-level Python web framework that encourages rapid development and clean design, ideal for building complex, database-driven websites.

- **Streamlit:** An open-source app framework specifically designed for creating and sharing data applications. It allows for the rapid development of interactive web apps with minimal code.

My Choice: Streamlit is chosen for its simplicity and efficiency in developing interactive data applications, making it ideal for ML model visualization and deployment.

2.4.5 CI/CD Tools

Continuous Integration and Continuous Deployment (CI/CD) tools automate the building, testing, and deployment of applications, enhancing development efficiency.

- **Jenkins:** An open-source automation server that supports building, deploying, and automating any project. It offers a vast plugin ecosystem but requires manual setup and maintenance.
- **GitHub Actions:** A CI/CD platform integrated with GitHub repositories, enabling automation of workflows directly from the repository. It's user-friendly and requires minimal setup.

MY Choice: GitHub Actions is selected for its seamless integration with GitHub repositories, simplifying the automation of workflows and reducing the overhead of managing separate CI/CD infrastructure.

2.4.6 Workflow Orchestration

Workflow orchestration tools manage complex ML pipelines, ensuring tasks are executed in the correct order and dependencies are handled appropriately.

- **ArgoCD:** A declarative, GitOps continuous delivery tool for Kubernetes. It automates the deployment of desired application states in specified target environments by continuously monitoring running applications and comparing the current, live state against the desired target state as specified in the Git repository.
- **ArgoCD:Flux CD:** An open-source GitOps tool that synchronizes Kubernetes clusters with configuration stored in Git repositories. It supports multi-tenancy and integrates well with Helm, making it suitable for managing complex deployments.
- **Jenkins X:** An extension of Jenkins designed for Kubernetes, providing automated CI/CD for cloud-native applications. It incorporates GitOps principles and supports preview environments for pull requests.
- **Spinnaker:** A multi-cloud continuous delivery platform that supports advanced deployment strategies like canary releases and blue/green deployments. It's suitable for managing deployments across various cloud providers.
- **Tekton:** A Kubernetes-native framework for creating CI/CD systems. It provides reusable and extensible pipelines, integrating seamlessly with other GitOps tools like Argo CD and Flux.

My Choice: ArgoCD is utilized for its GitOps approach to continuous delivery, providing automated deployment and synchronization of applications within Kubernetes clusters.

2.4.7 Pipeline Automation and Deployment Tools

Automating ML pipelines is crucial for enhancing efficiency, reproducibility, and scalability in ML workflows. (Kreuzberger, Kühl, and Hirschl 2023) provides an extensive evaluation of MLOps technologies, categorizing them based on their functionalities:

Table 4: MLOps Tools by Category

Category	Tools
Data Versioning	DVC, Pachyderm
Model Versioning	MLflow, ModelDB
Workflow Orchestration	Kubeflow Pipelines, Apache Airflow, Luigi
Model Serving	TensorFlow Serving, TorchServe, NVIDIA Triton Inference Server
Monitoring	Prometheus, Grafana, Seldon Core, Alibi Detect
CI/CD Integration	Jenkins, GitLab CI/CD, Git Actions , CircleCI
Infrastructure as Code	Terraform, Ansible, Helm
Experiment Tracking	MLflow, Weights & Biases, Comet.ml

This table illustrates the diverse ecosystem of tools available for implementing MLOps practices, each serving specific roles within the ML pipeline to ensure seamless integration and operation.

2.5 Challenges In MLOps, CRISP-DM and SEMMA

2.5.1 Challenges Associated with CRISP-DM and SEMMA Methodologies

Traditional methodologies like the Cross Industry Standard Process for Data Mining (CRISP-DM) and SEMMA (Sample, Explore, Modify, Model, Assess) have historically guided data mining and analytical projects effectively. However, with the advent of large-scale, dynamic, and complex machine learning applications, these approaches have exhibited critical shortcomings, particularly when scaling beyond experimental or exploratory projects.

Challenges Associated with CRISP-DM: CRISP-DM remains a widely adopted framework due to its clarity and comprehensive coverage of data mining phases. Nevertheless, specific limitations emerge clearly when applied in contemporary machine learning environments:

- **Linear Workflow Structure:** CRISP-DM defines a predominantly linear sequence—Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment. Although iterative loops exist theoretically, the practical implementation of feedback loops within CRISP-DM is minimal. This rigid linearity impedes responsiveness to real-time changes in data or business conditions, reducing adaptability and efficiency in dynamic operational settings.
- **Limited Emphasis on Post-Deployment Management:** CRISP-DM inadequately addresses continuous monitoring, model drift detection, or ongoing retraining post-deployment. The framework typically concludes with deployment viewed as the final step, overlooking the critical necessity of managing model performance proactively in real-world environments.

- **Manual and Resource-Intensive Processes:** The processes defined within CRISP-DM primarily emphasize manual tasks, which are not readily scalable. This manual focus can become resource-intensive and inefficient, particularly in scenarios involving high volumes of data, frequent retraining, or large-scale deployments.
- **Minimal Infrastructure Guidance:** CRISP-DM offers little explicit guidance on managing underlying computing resources, containerization, cloud integration, or continuous integration and continuous delivery (CI/CD) practices. This lack of structured infrastructure management guidance hinders scalability and robustness in production environments.

Challenges Associated with SEMMA: SEMMA methodology, primarily advocated by SAS Institute, offers a systematic data analysis structure. However, several critical limitations emerge clearly when addressing operational ML systems:

- **Lack of Business Understanding Phase:** SEMMA explicitly begins at the 'Sample' stage, omitting a formal business understanding phase. This omission can lead to misalignment between analytical outputs and strategic business objectives, significantly limiting the relevance and practical utility of analytical results.
- **Minimal Deployment and Maintenance Considerations:** SEMMA ends its workflow at the 'Assess' stage, without adequately addressing deployment, operational management, monitoring, and retraining. Such limitations result in poorly managed ML systems, where models degrade rapidly post-deployment due to lack of continuous performance oversight.
- **Inadequate Iterative and Feedback Loops:** While SEMMA proposes iteration primarily within exploratory analysis, it neglects broader iterative loops involving continuous integration of operational feedback, model retraining, or adjustment to data drift. Consequently, models may quickly become outdated and ineffective when data or operational conditions evolve.
- **Limited Scalability and Automation Support:** SEMMA provides minimal explicit provisions for scalability and automation in data processing and model lifecycle management. The methodology relies heavily on manual data exploration, modification, and model tuning, which becomes cumbersome and inefficient as project complexity and data volume increase.

These challenges clearly highlight critical gaps within CRISP-DM and SEMMA methodologies, especially in their application to modern machine learning operations (MLOps). The limitations underline the necessity of adopting MLOps practices, which address these challenges comprehensively through automation, continuous monitoring, collaborative workflows, and robust infrastructure management.

2.5.2 Challenges In MLOps

Machine Learning Operations (MLOps) has emerged as a critical discipline for deploying and maintaining machine learning models in production environments. However, organizations face several challenges in implementing effective MLOps practices. These challenges can be categorized into technical, organizational, and cultural domains.

Technical Challenges:

Technical challenges in MLOps pertain to the complexities involved in managing machine learning models throughout their lifecycle. Key technical challenges include:

- **Model Versioning:** Tracking different versions of models, datasets, and code is essential for reproducibility and auditing. Without proper versioning, it becomes difficult to reproduce results or understand the evolution of a model.
- **Reproducibility:** Ensuring that experiments and results can be consistently reproduced is vital for validating models and building trust. This requires meticulous tracking of data, code, configurations, and environments.
- **Consistent Performance:** Models may perform well in development but degrade in production due to data drift or changes in the environment. Monitoring and maintaining consistent performance is a continuous challenge.
- **Resource Consumption:** Training and deploying models, especially deep learning models, can be resource-intensive. Efficient resource management is necessary to optimize costs and performance.

These technical challenges necessitate robust tooling and infrastructure to manage the complexities of machine learning workflowseffectively.

Organizational Challenges

Organizational challenges arise from the need to integrate machine learning workflows into existing business processes and structures. Key organizational challenges include:

- **Cross-Functional Collaboration:** Effective MLOps requires collaboration between data scientists, engineers, operations teams, and business stakeholders. Aligning these diverse groups with different priorities and expertise can be challenging.
- **Managing Diverse Tools:** The MLOps ecosystem comprises a plethora of tools for data processing, model training, deployment, and monitoring. Integrating and managing these tools cohesively is complex.
- **Integrating ML Workflows with Software Processes:** Machine learning workflows often differ from traditional software development processes. Bridging this gap requires adapting existing processes or developing new ones to accommodate ML-specific needs.

Addressing these organizational challenges involves establishing clear communication channels, defining roles and responsibilities, and fostering a culture that supports machine learning initiatives.

Cultural Challenges

Cultural challenges pertain to the human aspects of adopting MLOps practices within an organization. Key cultural challenges include:

- **Resistance to Change:** Implementing MLOps often requires changes in workflows, tools, and mindsets. Employees may resist these changes due to comfort with existing processes or fear of the unknown.

- **Skill Gaps:** MLOps requires a combination of skills in machine learning, software engineering, and operations. Finding or developing talent with this interdisciplinary expertise can be difficult.
- **Stakeholder Education:** Educating stakeholders about the benefits and requirements of MLOps is crucial for securing buy-in and support. Lack of understanding can lead to misaligned expectations and priorities.

Overcoming cultural challenges involves investing in training, promoting awareness, and demonstrating the value of MLOps through successful projects.

Robustness and Resilience in ML Production Environments

Ensuring robustness and resilience in machine learning production environments is critical to maintaining reliable and reliable systems. Key considerations include

- **Robustness:** Models must perform reliably under various conditions, including noisy or adversarial inputs. Robustness ensures that models can handle real-world data variability. Employees may resist these changes due to comfort with existing processes or fear of the unknown.
- **Resilience:** The systems should be able to recover from failures and continue to operate effectively. This includes handling hardware failures, software bugs, and unexpected data issues.
- **Trustworthiness:** Building trust in AI systems involves ensuring transparency, fairness, and accountability. Users must have confidence in the decisions made by the machine learning models.

2.6 Frameworks and Maturity Models in MLOps

The adoption of Machine Learning Operations (MLOps) frameworks and maturity models is pivotal for organizations aiming to streamline their machine learning workflows and ensure scalable, reliable, and efficient deployment of ML models. These frameworks provide structured approaches to integrate ML systems into existing operations, while maturity models offer a roadmap for assessing and enhancing MLOps capabilities over time.

2.6.1 MLOps Frameworks

MLOps frameworks serve as blueprints for implementing best practices in the development, deployment, and maintenance of ML models. They encompass various components, including data processing, model training, validation, deployment, and monitoring. One such framework is presented by (John, Olsson, and Bosch 2021), which outlines the continuous development activities involved in MLOps and provides a structured approach to integrate these activities into organizational workflows.

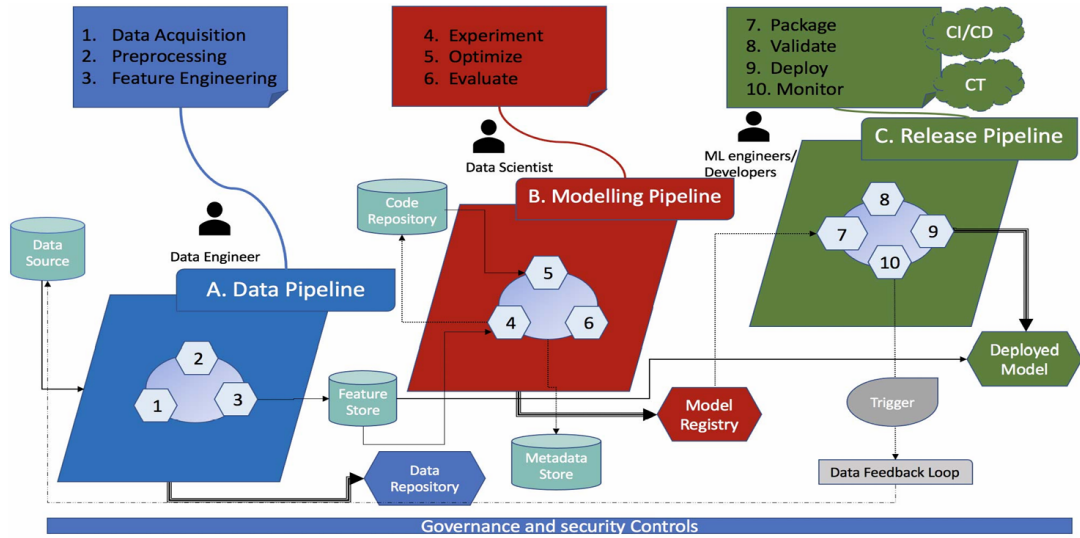


Figure 5: MLOps Framework, Source:(John, Olsson, and Bosch 2021)

This figure illustrates the iterative nature of MLOps processes, emphasizing the integration of continuous integration/continuous deployment (CI/CD) pipelines, automated testing, and monitoring to facilitate the seamless transition of ML models from development to production environments.

2.6.2 MLOps Maturity Models

Maturity models in MLOps provide organizations with a framework to assess their current capabilities and identify areas for improvement. These models typically define various levels of maturity, each characterized by specific practices, tools, and processes.

(John, Olsson, and Bosch 2021) proposes a maturity model comprising multiple levels, ranging from initial stages with ad-hoc processes to advanced stages featuring fully automated and integrated MLOps practices. This model assists organizations in benchmarking their MLOps maturity and strategizing their progression towards more sophisticated and efficient ML operations. This table delineates the progression through different ma-

Table 5: MLOps Maturity Levels

Maturity Level	Description
Level 0	Manual processes with minimal automation; lack of standardized workflows.
Level 1	Introduction of basic automation in model training and deployment.
Level 2	Implementation of CI/CD pipelines; improved collaboration between teams.
Level 3	Advanced automation with monitoring and feedback loops; scalable operations.
Level 4	Full integration of MLOps practices; continuous improvement and optimization.

turity levels, providing a roadmap for organizations to enhance their MLOps capabilities systematically.

2.6.3 Case Studies Highlighting MLOps Maturity

Empirical studies have validated the practical applicability of MLOps frameworks and maturity models. For instance, John et al. (2021) conducted case studies in three embedded systems companies, mapping their MLOps practices to the proposed maturity model. The findings demonstrated the model’s effectiveness in identifying gaps and guiding improvements in MLOps implementations.

Furthermore, a study by (Faubel and Schmid 2024) explored MLOps adoption in Industry 4.0 settings, analyzing multiple case studies to understand the challenges and strategies involved in implementing MLOps in industrial environments. The research highlighted the importance of tailoring MLOps practices to specific organizational contexts and the role of maturity models in facilitating this adaptation.

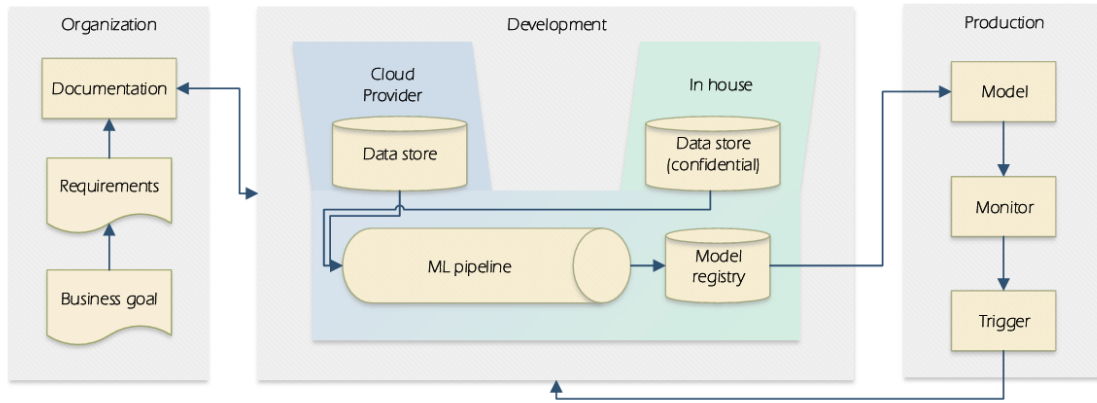


Figure 6: MLOps in Industry 4.0 Overview, Source: (Faubel and Schmid 2024)

This figure showcases the varying degrees of MLOps adoption across different industrial case studies, emphasizing the need for customized approaches based on organizational maturity and specific operational requirements.

2.7 Comparative Analysis of MLOps Approaches

2.7.1 Review and Comparative Analysis of case studies and Pararical Applications of MLOps

The adoption and implementation of Machine Learning Operations (MLOps) differ significantly across organizations and industries. Each implementation is influenced by various factors, including the organization’s size, domain-specific requirements, and available technological resources. In recent studies, various comparative analyses and case studies have highlighted both commonalities and divergences in the approaches organizations adopt towards MLOps.

(Faubel and Schmid 2024) conducted a detailed multiple case study focusing specifically on the implementation of MLOps within Industry 4.0 scenarios. The authors explored the practices adopted by three major European companies, identifying critical insights into their operational processes, organizational structures, toolsets, and technical environments. Their findings reveal a wide variance in practices adopted by different companies, reflecting the customized nature of MLOps depending on the specific industrial context,

resources, and data environments. However, despite these variances, certain commonalities were also observed, such as dedicated MLOps teams, hybrid usage of open-source and commercial tools, and clearly defined processes for ML model deployment.

2.7.2 Categorization of MLOps Practices and Suitability for Various Project Types

Understanding the suitability of specific MLOps practices for different project types, especially complex domains such as Computer Vision (CV), requires a detailed evaluation of available tools and methodologies.

(Diaz-De-Arcaya et al. 2023) offer a comprehensive systematic literature review (SLR) comparing various MLOps and AIOps methodologies, tools, and their suitability for specific project contexts. This analysis provides an in-depth overview that helps stakeholders select appropriate tools and practices aligned with their specific project needs and organizational context.

The authors further elaborate a comparative framework, explicitly outlining the differences and overlaps between MLOps and AIOps paradigms. This comparative framework is detailed in Table ,as follows:

Table 6: Comparison Framework between the MLOps and AIOps Paradigms (Source: (Diaz-De-Arcaya et al. 2023))

Aspect	MLOps	AIOps
Inception	2020	2019
Concept	Machine Learning Operations	AI for IT Operations
Goal	Creating stable products from ML prototypes	Address complexity in modern IT systems, reduce human intervention
Enablers	CI/CD, orchestration, reproducibility, collaboration, continuous monitoring	Big Data, cloud computing, observability
Focus	Multidisciplinary	Information Technology
Use Cases	Failure prediction, autonomous vehicles, defect detection, electricity market	Spikes in demand, maximize uptime, optimize environments, improve efficiency

This comparative framework clearly outlines fundamental distinctions and practical overlaps between MLOps and AIOps. It provides valuable guidance for organizations to identify which approach aligns better with their operational needs and technological infrastructures.

2.7.3 Insights From Paratical Implementation

Insights from practical implementations further highlight the contextual differences across various industrial sectors. For example, case studies in Industry 4.0 by (Faubel and Schmid 2024) indicate that MLOps architectures typically integrate components for data management, processing, training, testing, deployment, and monitoring. The architectures are structured into layers such as tool stacks, infrastructure, and execution environments, each managed by specialized teams. These insights underline the necessity for clearly defined architectural guidelines and standardized yet flexible frameworks to effectively implement MLOps.

Furthermore, (Diaz-De-Arcaya et al. 2023) emphasize the importance of automation and orchestration tools within the MLOps domain, including Kubernetes, MLflow, Kubeflow, and Jenkins, to manage complex ML workflows. These tools support the continuous integration, deployment, and monitoring processes essential to maintain efficient and robust ML pipelines.

2.7.4 Specific Considerations For Computer Vision Projects

Computer Vision (CV) projects particularly benefit from tailored MLOps practices due to the complexity and data-intensive nature of CV workloads. Continuous training (CT), robust data pipelines, and effective model monitoring are particularly critical in CV contexts to handle high volumes of visual data and rapid model degradation issues. Tools like TensorFlow Extended (TFX), Kubeflow, and MLflow are frequently recommended to manage and automate CV-specific pipelines effectively, ensuring reproducibility and reliability in model performance.

(Diaz-De-Arcaya et al. 2023) highlights the significant role played by CI/CD practices specifically adapted to manage CV workflows, demonstrating how CV-specific MLOps implementations ensure rapid and effective model updates and deployments. Such tailored approaches highlight the adaptability and necessity of context-specific MLOps methodologies.

2.8 Sustainability and Future Directions For MLOps

2.8.1 Sustainable Practices in MLOps

As the use of Machine Learning (ML) expands across industries, ensuring the sustainability of MLOps practices becomes imperative. Sustainability in MLOps refers not only to ecological and energy-efficient model training and deployment but also to long-term maintainability, ethical design, and responsible AI governance (Tamburri 2020). Effective sustainable MLOps frameworks aim to minimize the resource footprint of ML models while supporting reproducibility, explainability, and robustness. Sustainable practices in MLOps encompass:

- Efficient resource utilization (e.g., model compression, pruning, quantization).
- Automation of retraining pipelines, minimizing human and computational cost.
- Reusability and modularization of ML pipeline components.
- Monitoring for bias and fairness, especially in models deployed long-term.
- Versioning and audit trails to improve traceability and reduce technical debt.

(Tamburri 2020) argues that sustainability in MLOps is closely tied to trustworthy AI principles, which include robustness, security, human oversight, and accountability. The goal is to evolve MLOps into a system that is not only functional but also aligned with long-term social and environmental values.

2.8.2 Emerging Trends and Challenges

The field of MLOps is undergoing rapid evolution, with several notable trends gaining traction:

- Shift toward Edge and Federated MLOps: Organizations increasingly adopt edge computing and federated learning, demanding decentralized MLOps solutions.
- Integration with Responsible AI: Practices such as explainability, fairness auditing, and AI ethics are being embedded into MLOps pipelines.

- Use of Large Language Models (LLMs): The rise of foundation models has increased demand for scalable and low-latency MLOps systems.
- AutoMLOps: There is growing adoption of AutoML tools combined with MLOps automation (AutoMLOps), which further abstracts and automates the development-to-deployment flow.
- Green AI awareness: Organizations are now tracking and attempting to minimize the carbon footprint of training and inference tasks, leading to optimization at the infrastructure and algorithmic level.

2.8.3 Research Gap and Future Opportunities

Despite rapid growth, the MLOps landscape remains fragmented and under-researched in many areas. Several gaps and opportunities have been identified:

Table 7: Research Opportunities in MLOps Gap Areas

Gap Area	Research Opportunity
Integration of Ethics into Pipelines	Developing frameworks for real-time fairness and bias monitoring
Toolchain Interoperability	Creating standardized interfaces between orchestration tools
Low-resource MLOps	Methods to run full pipelines on edge/IoT devices
Lifecycle Traceability	Efficient provenance tracking across data, model, and config versions
Energy-Efficient MLOps	Optimization algorithms for training and inference to reduce power use

(Tamburri 2020) emphasizes that while frameworks and pipelines have matured, sustainability principles are not yet embedded into mainstream MLOps practices. Research into MLOps maturity, especially in smaller organizations or low-resource environments, is still lacking.

2.9 Summary and Conclusion

The reviewed literature presents a comprehensive understanding of the evolution, implementation, challenges, and future directions of Machine Learning Operations (MLOps). Across various domains and case studies, a consistent theme emerges—MLOps has become a critical enabler for the successful operationalization of machine learning models in dynamic, real-world environments.

MLOps integrates the principles of DevOps with the specific demands of ML workflows, introducing practices such as Continuous Integration (CI), Continuous Delivery (CD), and Continuous Training (CT). These practices address the unique complexities of ML systems, including model drift, data volatility, reproducibility issues, and high computational costs. The iterative and cyclical nature of MLOps phases ensures continuous feedback, leading to better model performance and reliability over time (Symeonidis et al. 2022; Zhou, Yu, and Ding 2020).

The literature demonstrates that MLOps frameworks and maturity models are essential for organizations to benchmark and structure their implementation. Tools such as MLflow, Kubeflow, and TFX provide end-to-end support for pipeline management, and industry-grade maturity models (e.g., those by Google and Microsoft) help assess organizational readiness for full-scale MLOps adoption (Kreuzberger, Kühl, and Hirschl 2023; John, Olsson, and Bosch 2021).

Moreover, the literature emphasizes the growing relevance of sustainability and ethical considerations in MLOps. Research by (textcite(tamburri2020sustainable)) underlines the importance of trustworthy, energy-efficient, and explainable AI systems. Similarly, the studies by (Faubel and Schmid 2024) and (Diaz-De-Arcaya et al. 2023) indicate that contextual adaptation and domain-specific tailoring of MLOps practices — particularly in areas like Computer Vision and Industry 4.0—are vital for successful deployments. Despite the advancements, notable gaps remain. There is a clear need for further research into:

- Automating retraining and monitoring triggers with minimal human intervention.
- Ensuring interoperability across diverse MLOps toolchains.
- Enabling sustainable, scalable, and low-resource MLOps pipelines, especially in edge and federated settings.

In conclusion, MLOps is a rapidly evolving field with profound implications for the scalability, sustainability, and trustworthiness of AI systems. An incremental and iterative approach, guided by established frameworks and adapted to organizational needs, is essential for long-term success. Further exploration into automation, cross-platform tool compatibility, and responsible AI integration will be instrumental in shaping the future of MLOps.

3 Methodology and Analysis Procedure

3.1 Overview Methodology

This project implements an advanced, systematic approach for developing and deploying a machine learning application, utilizing several modern technologies and frameworks to streamline the process from data preprocessing and model training to final deployment. The primary goal is to automate and operationalize machine learning workflows to ensure efficiency, reliability, and reproducibility in production environments.

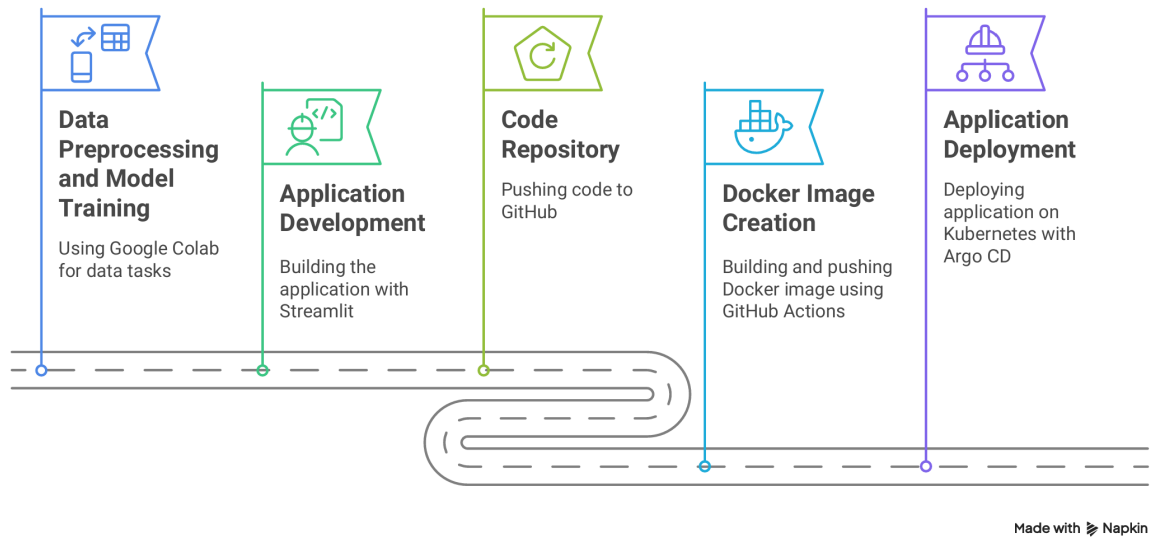


Figure 7: Proposed Methodology Diagram

Figure 7 presents the proposed methodology framework, illustrating the structured workflow adopted in this project.

3.2 Data preprocessing and Model Training

The initial stages of data preprocessing and model training are conducted using Google Colab, a cloud-based platform that provides the required computational resources and collaborative features. The data preprocessing phase involves several steps:

- **Data Acquisition:** Collection and integration of data from various sources, ensuring the quality and consistency of the dataset.
- **Data Cleaning and Preparation:** Handling missing values, outlier detection, and normalization or standardization of data.
- **Feature Engineering:** Identifying and extracting relevant features from raw data, transforming them into a format suitable for machine learning algorithms.

After preprocessing, the machine learning model training commences, including:

- **Model Selection and Evaluation:** Exploration and selection of appropriate machine learning algorithms based on performance metrics such as accuracy, precision, recall, and F1-score.
- **Hyperparameter Tuning:** Optimization of the model parameters using techniques like Grid Search and Randomized Search to enhance model performance.
- **Validation:** Implementation of cross-validation methods to ensure the robustness and generalizability of the trained model.

3.3 Application Development With Streamlit

Once the model is trained and validated, the application is developed using Streamlit, an open-source framework ideal for building interactive web applications for data science and machine learning projects. The steps include:

- **User Interface (UI) Design:** Creation of a user-friendly, interactive web interface to display model predictions, visualizations, and insights effectively.
- **Integration with Model:** Seamless integration of the trained machine learning model into the Streamlit application, allowing real-time predictions based on user inputs.
- **Testing and Debugging:** Rigorous testing of the application to ensure functionality, responsiveness, and reliability across different user scenarios.

3.4 Version Control and Collaboration via GitHub

The source code, datasets, and related documentation are maintained using GitHub, providing version control and enabling effective collaboration among team members. The practices followed include:

- **Branching Strategy:** Use of branches to manage parallel development efforts, feature implementation, and hotfixes.
- **Code Reviews and Quality Checks:** Regular peer reviews and automated testing pipelines to maintain code quality and identify potential issues early.
- **Continuous Integration (CI):** Automated build and testing processes to integrate code changes frequently, ensuring stability and consistency.

3.5 Containerization and Deployment Automation with Docker and GitHub Actions

The application is containerized using Docker, ensuring consistency across various deployment environments by encapsulating the application and its dependencies into containers. The deployment workflow includes:

- **Dockerization:** Creation of Docker images that encapsulate the application, its dependencies, and runtime environment.

- **GitHub Actions:** Setting up automated workflows on GitHub Actions to build Docker images and push them to Docker Hub or other container registries upon code changes.

3.6 Kubernetes Deployment and Argo CD

The final deployment phase leverages Kubernetes, an open-source orchestration tool, for managing containerized applications. Argo CD, a declarative continuous delivery tool, ensures synchronization between the GitHub repository and the Kubernetes cluster, automating application deployment. The deployment steps include:

- **Kubernetes Cluster Setup:** Configuration of a robust Kubernetes cluster suitable for hosting production-grade applications.
- **Argo CD Integration:** Implementation of Argo CD to continuously monitor the GitHub repository, deploying any changes automatically to the Kubernetes cluster.
- **Monitoring and Logging:** Setting up monitoring tools (e.g., Prometheus, Grafana) and logging systems to track the application's performance and reliability

3.7 Analysis and Optimization

Post-deployment, the application undergoes continuous monitoring and optimization:

- **Performance Monitoring:** Tracking performance metrics, identifying bottlenecks, and conducting regular performance tuning.
- **Feedback Loop:** Incorporating user feedback and system monitoring data into the iterative improvement process, ensuring that the application remains aligned with user needs and expectations.

By adopting this structured, automated, and collaborative approach, the project ensures robust, scalable, and efficient deployment of machine learning applications, significantly reducing manual interventions and potential errors in the deployment process.

4 Project Development

The development of this violence detection system was executed through a rigorously structured and modularized pipeline. The workflow began with acquiring and preprocessing a labeled dataset suitable for video classification, followed by implementing and training two distinct model architectures: a Convolutional Neural Network (CNN) coupled with Long Short-Term Memory (LSTM) layers for spatiotemporal learning, and a Transformer-based architecture (TimeSformer) for enhanced attention-based modeling of video frames. Post model training, the most performant model was integrated into a user-facing web application using Streamlit. This application was containerized using Docker to ensure consistency across environments and facilitate scalable deployment. Furthermore, the application was orchestrated via Kubernetes using manifest files (deployment.yaml and service.yaml), and automated deployment and versioning were handled through GitHub Actions integrated with ArgoCD. This comprehensive MLOps

approach ensures modularity, reproducibility, and maintainability, embodying modern best practices in AI-driven application development.

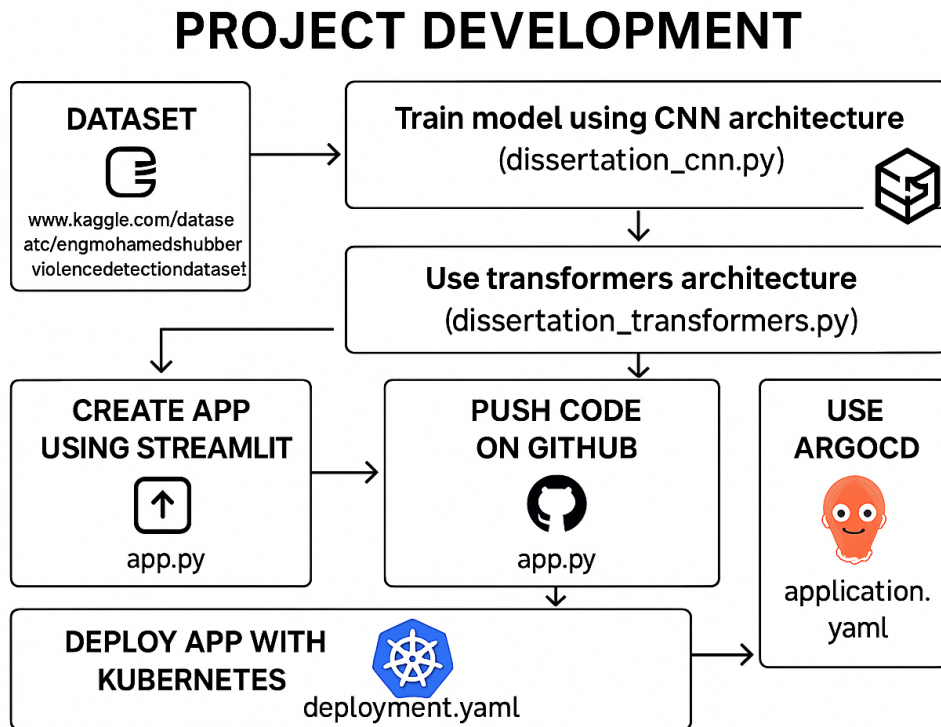


Figure 8: Project Workflow

4.1 Data Acquisition

The dataset used for this project was sourced from Kaggle: Violence Detection Dataset. It consists of video clips labeled as "Violent" or "Non-Violent" and was instrumental in training both CNN and Transformer-based models.

4.2 Initial Model Training with CNN

The first model iteration was developed using a custom-designed Convolutional Neural Network (CNN) architecture, implemented in the file `dissertation-cnn.py`. This architecture served as a baseline and was designed to process sequences of video frames (16 per sample) with a resolution of 64x64 pixels and 3 color channels (RGB). The architecture consisted of the following layers:

- A 3D convolutional layer with 32 filters and a kernel size of (3, 3, 3), using ReLU activation.
- A 3D max-pooling layer with a pool size of (2, 2, 2) to downsample the spatial and temporal dimensions.
- A flattening layer to convert the 3D tensor output into a 1D vector.
- A dense (fully connected) hidden layer with 64 neurons and ReLU activation.

- A final dense output layer with 2 neurons (for binary classification: "Violent" or "Non-Violent") using softmax activation.

```
def build_cnn(input_shape):
    """
    Custom CNN for spatial feature extraction.
    """
    model = Sequential([
        Conv2D(64, (3, 3), activation='relu', padding='same',
input_shape=input_shape),
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(128, (3, 3), activation='relu', padding='same'),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(256, (3, 3), activation='relu', padding='same'),
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Flatten()
    ])
    return model
```

Listing 1: CNN Model

Preprocessing steps included frame extraction from each video, resizing to 64x64, normalization, and padding (if fewer than 16 frames were present). The model was compiled using the Adam optimizer and categorical cross-entropy loss function, and trained over 20 epochs with a batch size of 8. Performance was evaluated on a validation split from the training data, and the CNN model reached moderate accuracy levels, indicating potential for further enhancement with more complex architectures.

4.3 Transformer-Based Model Implementation

After establishing a baseline with the CNN architecture, the next phase of development focused on adopting a more sophisticated and state-of-the-art model capable of capturing complex spatio-temporal relationships in video data. The model chosen for this

was the TimeSformer (Time-Space Transformer), a specialized Vision Transformer (ViT) designed for video understanding tasks. The implementation was done in `dissertation-transformers.py`. The TimeSformer architecture processes video sequences by dividing each frame into patches and applying self-attention across both spatial and temporal dimensions. This enables it to learn both motion dynamics and object appearance more effectively than traditional CNN-based methods. Key hyperparameters and architecture details include:

- `image-size=64` : Input frames are resized to 64x64 pixels.
- `num-frames=16` : Each video input consists of a sequence of 16 frames.
- `num-classes=2` : Binary classification (Violent vs. Non-Violent).
- `dim=512` : Hidden dimension of the transformer.
- `depth=4` : Number of transformer encoder layers.
- `heads=8` : Number of self-attention heads.
- `patch-size=16` : Frames are divided into non-overlapping 16x16 patches.
- `attn-dropout=0.1` : Dropout rate for attention layers.
- `ff-dropout=0.1` : Dropout rate for feed-forward layers.

```
model = TimeSformer(  
    image_size=64,  
    num_frames=X_train.shape[1],  
    num_classes=len(torch.unique(y_train)),  
    dim=512,  
    depth=4,  
    heads=8,  
    patch_size=16,  
    attn_dropout=0.1,  
    ff_dropout=0.1)
```

Listing 2: Transformer Model

Training was conducted using the same preprocessing pipeline as the CNN model: extracting frames, resizing, normalization, and padding sequences to maintain consistent input shape. The model was trained using Adam optimizer and a learning rate scheduler, with a cross-entropy loss function optimized for classification. Batch size was set to 4, and early stopping was applied based on validation loss.

The TimeSformer demonstrated a significant boost in classification performance, with improved generalization and robustness across various types of violent and non-violent video samples. Its performance metrics, including precision, recall, and F1-score, were substantially higher than the CNN model, validating the suitability of transformers for video-based action recognition tasks.

4.4 Streamlit App Development

To operationalize the trained model and enable accessible predictions via a user-friendly interface, a web application was developed using Streamlit (app-py). Streamlit was chosen for its ease of use, minimal setup, and ability to quickly turn Python scripts into interactive web apps.

The application consists of the following major components:

- **Video Upload Interface:** Allows users to upload videos in common formats like MP4, AVI, MOV, or MKV. Uploaded videos are temporarily stored for processing.
- **Frame Extraction and Preprocessing:** Once a video is uploaded, it is processed by extracting frames at fixed intervals. The frames are then resized to 64x64 pixels, normalized, and padded (if necessary) to create a fixed-size tensor (16 frames per video), consistent with the input expected by the Transformer model.
- **Model Loading and Inference:** The pre-trained TimeSformer model is loaded using a dedicated function. After preprocessing the video frames, they are passed through the model to predict the probability distribution across classes.
- **Prediction Display:** The application displays the predicted class label ("Violent" or "Non-Violent") along with the confidence score for each class. The confidence scores are shown using a bar chart for better interpretability.
- **Error Handling and Feedback:** Users are notified if their input format is incorrect or if an error occurs during processing, ensuring robustness and a smooth user experience.

Below is an excerpt of the relevant Streamlit code:

```
# Streamlit App
def main():
    st.title("Violence Detection (Transformer Model)")
    st.write("Upload a video to classify it as **Non-Violent** or **Violent**.")

    video_file = st.file_uploader("Upload a Video", type=["mp4", "avi", "mov", "mkv"])

    if video_file is not None:
        temp_video_path = "temp_video.mp4"
        with open(temp_video_path, "wb") as f:
            f.write(video_file.read())

        st.video(temp_video_path)

        model = load_trained_model()

        with st.spinner("Analyzing the video..."):
            predicted_class, probabilities = predict_video_class(
                temp_video_path, model)

        if predicted_class is not None:
            st.success(f"Prediction: **{predicted_class}**")
            st.write("Confidence Scores:")
```

```

for i, class_name in enumerate(CLASSES_LIST):
    st.write(f"{class_name}: {probabilities[i]:.2f}")

if os.path.exists(temp_video_path):
    os.remove(temp_video_path)

```

Listing 3: Streamlit App

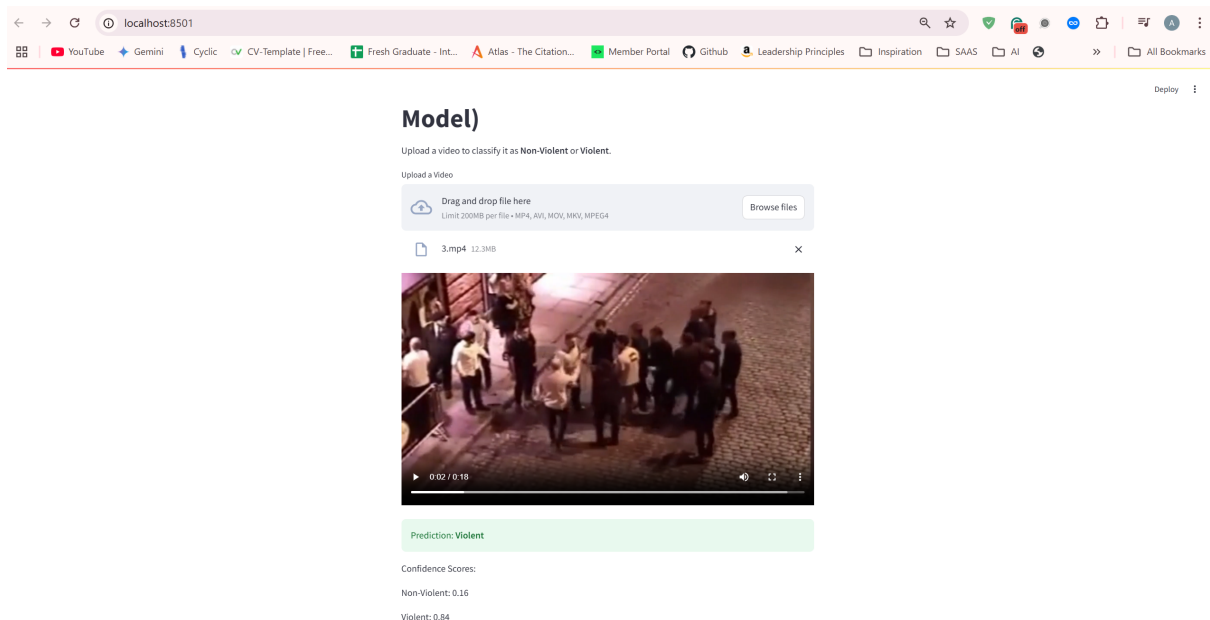
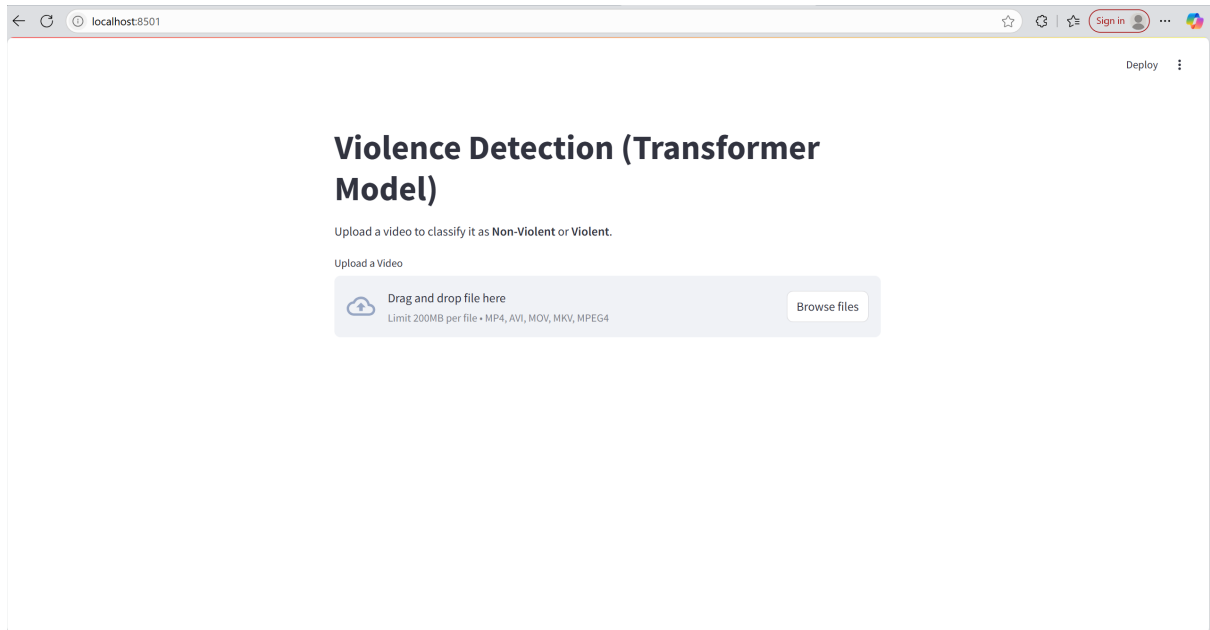


Figure 9: Streamlit App

This real-time, browser-based interface enabled the seamless demonstration and usage of the violence detection model for both developers and end-users without requiring knowledge of machine learning or backend deployment.

4.5 Version Control and GitHub Integration

Version control was implemented using Git and hosted on GitHub to facilitate effective collaboration, code versioning, and continuous integration workflows. Two separate repositories were maintained for modularity and clarity of responsibilities: one for the application codebase (including model scripts like `dissertation-cnn.py`, `dissertation-transformers.py`, the preprocessing utilities, and the Streamlit application `app.py`) and a second repository exclusively for Kubernetes manifest files, including `deployment.yaml` and `application.yaml`. This separation allowed for cleaner CI/CD pipelines and better management of infrastructure-as-code (IaC).

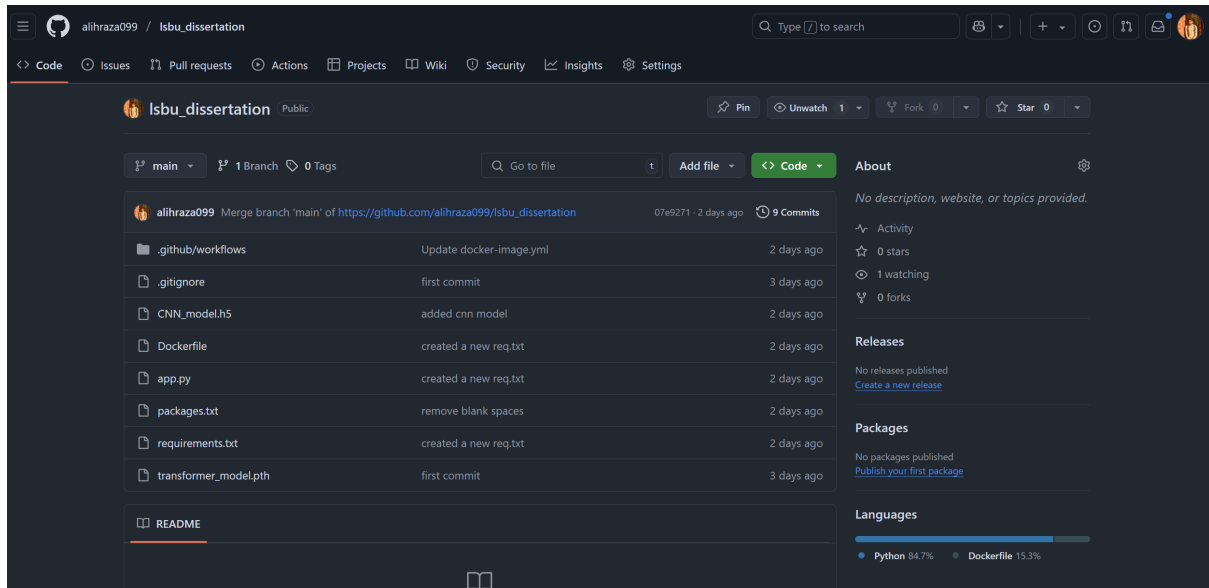


Figure 10: GitHub Repo

4.6 Dockerization

To ensure the application could be deployed consistently across different environments and platforms, Docker was employed to containerize the entire Streamlit application. Containerization not only simplifies deployment but also encapsulates all dependencies and system-level configurations required for running the app. The Docker image for the application is automatically built using GitHub Actions as part of the CI/CD pipeline defined in the app code repository.

A dedicated GitHub Actions workflow is triggered on every commit to the main branch. This workflow performs the following steps:

- Checks out the codebase i.e `app.py`, `Dockerfile`, `requirements.txt`.
- Builds the Docker image from the `Dockerfile`.
- Tags and pushes the image to a container registry (Docker Hub).

This automated Docker build process ensures that the container image always reflects the latest application code, maintaining consistency between development and deployment environments.

A `Dockerfile` was created with the following structure:

```

# Dockerfile for Streamlit app with OpenCV
FROM python:3.10-slim

# Set working directory
WORKDIR /app

# Install system dependencies needed for OpenCV and Streamlit
RUN apt-get update && \
    apt-get install -y \
        ffmpeg \
        libsm6 \
        libxext6 \
        libgl1 \
        libglu1-mesa-dev \
        libglu1-mesa \
        apt-get clean && \
    rm -rf /var/lib/apt/lists/*

# Copy and install Python packages
COPY requirements.txt .
RUN pip install --upgrade pip && pip install --no-cache-dir -r
requirements.txt

# Copy app files
COPY . .

# Expose Streamlit's default port
EXPOSE 8501

# Start Streamlit app
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--
server.address=0.0.0.0", "--server.enableCORS=false"]

```

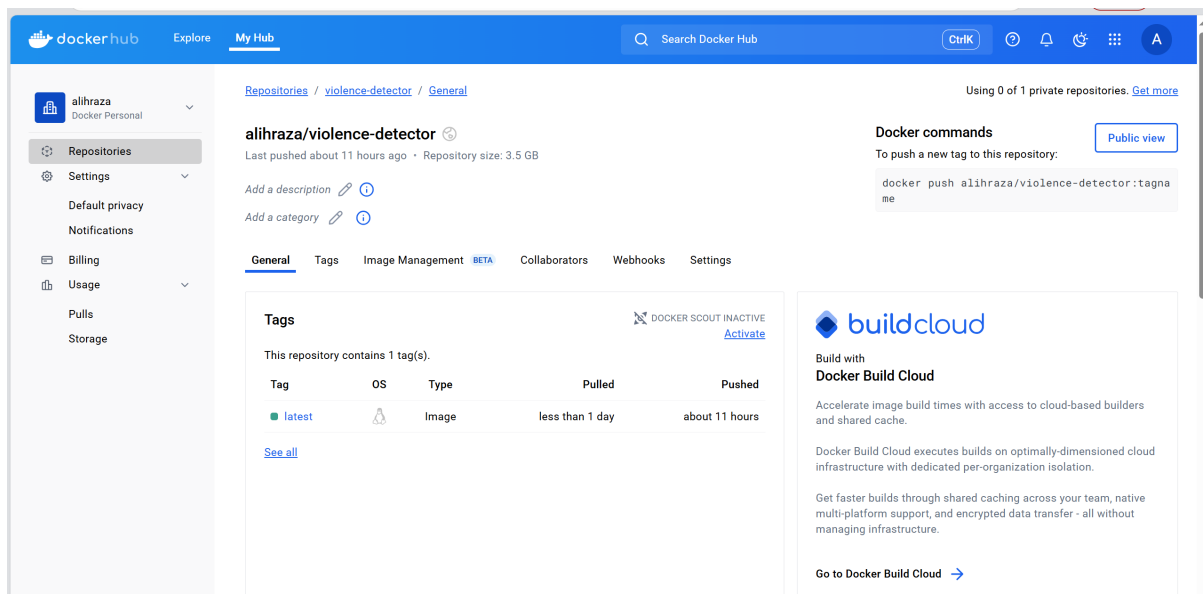


Figure 11: Docker Hub Image

4.7 Kubernetes Deployment

To orchestrate deployment and scaling of the containerized Streamlit application, Kubernetes was employed. A single Kubernetes manifest file (deployment. yam]) was written to manage both the deployment and service configuration, allowing for streamlined management and easier synchronization through GitOps workflows. This combined file defined the application specifications, the Docker image to be used, and the port exposure details for local access via Minikube. The deployment.yaml file included a Deployment resource with one replica and a Service resource of type NodePort to make the application accessible externally.

Below is the configuration:

```
    apiVersion: apps/v1
kind: Deployment
metadata:
  name: violence-detector
spec:
  replicas: 1
  selector:
    matchLabels:
      app: violence-detector
  template:
    metadata:
      labels:
        app: violence-detector
    spec:
      containers:
        - name: violence-detector
          image: alihraza/violence-detector:latest
          ports:
            - containerPort: 8501
          resources:
            requests:
              memory: "128Mi"
              cpu: "250m"
            limits:
              memory: "256Mi"
              cpu: "500m"
---

apiVersion: v1
kind: Service
metadata:
  name: violence-detector-service
spec:
  type: NodePort
  selector:
    app: violence-detector
```

```
ports:
- protocol: TCP
  port: 80
  targetPort: 8501
```

Minikube was used to test Kubernetes deployment locally. The Docker image was built and pushed to Docker Hub using GitHub Actions, and then pulled directly into the Minikube cluster during deployment. To apply the manifest:

```
kubectl apply -f deployment.yaml
```

Listing 4: Command for Deployment

The application was exposed using a NodePort defined in the same deployment.yaml, and accessed via the following command:

```
minikube service violence-detection-s rvice
```

Listing 5: Command to run the app and expose it for localhost

```
PS E:\LSBU\Disertation\Development> kubectl get pods --all-namespaces
NAMESPACE      NAME                                                    READY   STATUS    RESTARTS   AGE
argocd          argocd-application-controller-0                       1/1     Running   1 (3m54s ago)  9h
argocd          argocd-applicationset-controller-86848bf96-sc6r1       1/1     Running   2 (2m5s ago)   9h
argocd          argocd-dex-server-66b6b6866-dxbzs                    1/1     Running   1 (3m53s ago)  9h
argocd          argocd-image-updater-d568c7d86-5rd2g                 1/1     Running   1 (3m55s ago)  8h
argocd          argocd-notifications-controller-667fc74bbf-7lkpr      1/1     Running   2 (3m15s ago)  9h
argocd          argocd-redis-6c6cfdbf55-vr9rb                        1/1     Running   1 (3m54s ago)  9h
argocd          argocd-repo-server-5588567b85-525fx                  1/1     Running   1 (3m55s ago)  9h
argocd          argocd-server-578fc9fdbb-4n42f                       1/1     Running   1 (3m55s ago)  9h
default         violence-detector-5d97868696-7z2s9                    1/1     Running   2 (2m4s ago)   9h
kube-system     coredns-668d6bf9bc-mvwvj                             1/1     Running   7 (2m1s ago)   91d
kube-system     etcd-minikube                                          1/1     Running   7 (2m5s ago)   91d
kube-system     kube-apiserver-minikube                              1/1     Running   7 (115s ago)   91d
kube-system     kube-controller-manager-minikube                     1/1     Running   8 (68s ago)    91d
kube-system     kube-proxy-nshjg                                      1/1     Running   7 (2m4s ago)   91d
kube-system     kube-scheduler-minikube                              1/1     Running   7 (2m5s ago)   91d
kube-system     storage-provisioner                                  1/1     Running   18 (89s ago)   91d

PS E:\LSBU\Disertation\Development> kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP    9h
violence-detector-service NodePort    10.98.56.23   <none>         80:30641/TCP 9h

PS E:\LSBU\Disertation\Development> minikube service violence-detector-service
|-----|
| NAMESPACE | NAME                | TARGET PORT | URL                |
|-----|
| default   | violence-detector-service | 80          | http://192.168.49.2:30641 |
|-----|
🚀 Starting tunnel for service violence-detector-service.
|-----|
| NAMESPACE | NAME                | TARGET PORT | URL                |
|-----|
| default   | violence-detector-service |            | http://127.0.0.1:49884 |
|-----|
🌐 Opening service default/violence-detector-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Figure 12: Kubernetes Deployment and Service

This setup simulates a production-like environment and validates the containerized application's readiness for deployment in cloud platforms such as GKE,EKS,or AKS.

4.8 CI/CD with GitHub Actions and ArgoCD

To establish a robust and automated deployment pipeline, CI/CD was implemented using GitHub Actions in combination with ArgoCD. GitHub Actions Workflow A workflow YAML file was added to the application repository under • github/workflows/docker-image.yml. This pipeline is triggered on every push to the main branch. It automates the process of building the Docker image and pushing it to Docker Hub. Below is an excerpt of the workflow:

```
name: Build and Push Docker Image

on:
  push:
    branches: [ main ]
    paths:
      - 'app.py'
      - 'requirements.txt'
      - 'Dockerfile'

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to DockerHub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }

      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: alihraza/violence-detector:latest
```

The workflow uses GitHub Secrets to securely handle Docker Hub credentials. The resulting image is stored in Docker Hub and referenced in the Kubernetes deployment.yaml

manifest.

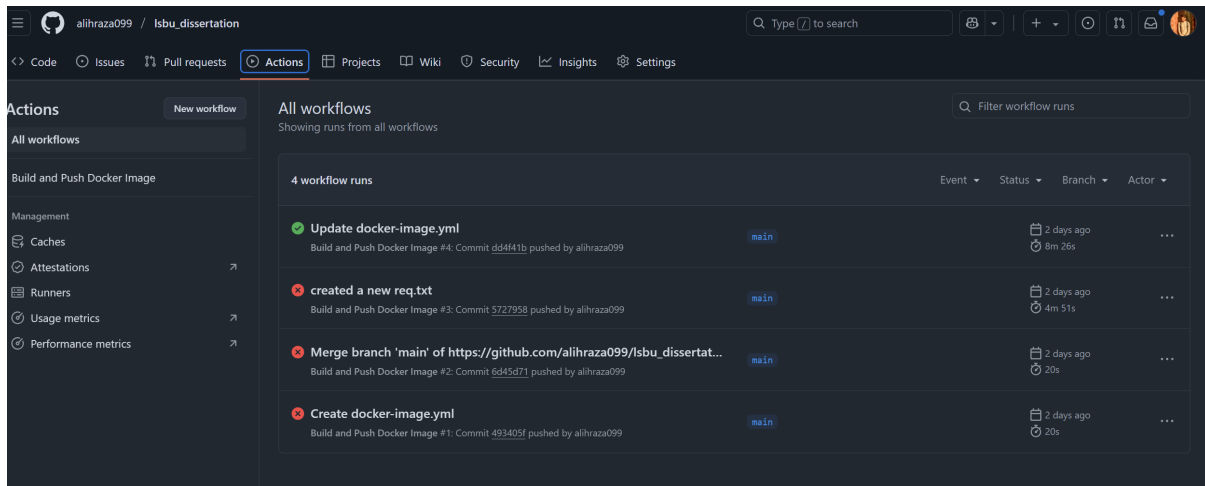


Figure 13: GitHub Actions

ArgoCD Integration

ArgoCD was configured to monitor the Kubernetes manifests repository hosted on GitHub. The repository includes the combined deployment.yaml, which defines both the Deployment and Service resources.

Additionally, ArgoCD is integrated with the ArgoCD Image Updater tool. This tool automatically checks for the latest image version in Docker Hub and updates the image reference in the manifest file, using the following configuration in the Application-manifest:

```
    apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: violence-detector
  namespace: argocd
  annotations:
    argocd-image-updater.argoproj.io/image-list:
      ↪ violence-detector=alihraza/violence-detector
    argocd-image-updater.argoproj.io/violence-detector.update-strategy:
      ↪ tegy: latest
spec:
  project: default
  source:
    repoURL: https://github.com/alihraza099/dissertation_k8s.git
    targetRevision: main
    path: .
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
```

```
selfHeal: true
```

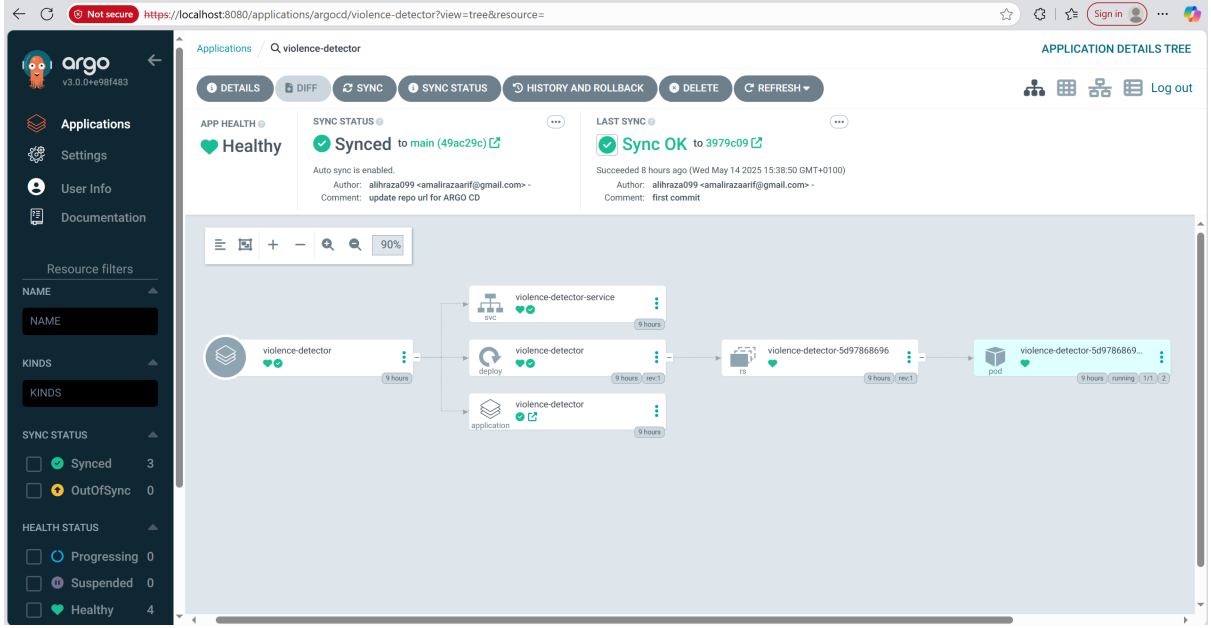


Figure 14: ArgoCD Integration

This configuration ensures ArgoCD will automatically redeploy the application whenever either a new Docker image is published to Docker Hub or when changes are pushed to the main branch of the manifests repository. This comprehensive setup offers full GitOps automation, guarantees traceability of deployments, and minimizes manual intervention.

5 Conclusion

This dissertation has thoroughly examined the incremental development approach within the context of Machine Learning Operations (MLOps), emphasizing its effectiveness in streamlining the development, deployment, and management of machine learning systems, specifically within computer vision applications. By critically evaluating the literature and synthesizing insights from practical implementations, the research has clearly demonstrated that adopting MLOps principles significantly enhances the operational reliability, scalability, and continuous improvement of ML models in production environments.

Key findings from this study underscore the critical necessity of integrating continuous integration (CI), continuous delivery (CD), and continuous training (CT) into machine learning workflows. These practices not only automate the ML lifecycle but also enable organizations to respond swiftly to evolving data patterns and business requirements. Furthermore, the study has highlighted the importance of robust model governance, continuous monitoring, and systematic retraining to maintain model accuracy and performance over time.

The practical component of this research provided a clear demonstration of MLOps by employing industry-standard tools such as Kubernetes for container orchestration, GitHub for version control, Docker for containerization, and ArgoCD for workflow automation.

This hands-on approach has validated the theoretical underpinnings discussed in the literature review, illustrating that structured, automated, and collaborative practices significantly reduce manual intervention and improve model deployment reliability.

However, the dissertation also identified ongoing challenges within the MLOps domain, such as scalability issues, maintaining comprehensive governance and compliance, and addressing cultural and organizational resistance. The findings indicate that successfully overcoming these barriers requires both technological advancement and cultural shifts within organizations to foster interdisciplinary collaboration and continuous learning.

Looking forward, the future of MLOps will likely involve greater automation, enhanced interoperability among toolchains, and more sustainable, energy-efficient practices. Continued research is needed to explore these emerging trends further and address the remaining gaps, particularly regarding the integration of responsible AI principles into MLOps workflows.

In conclusion, adopting an incremental development approach in MLOps is essential for realizing the full potential of machine learning, particularly in complex domains such as computer vision. Organizations embracing these practices can expect to achieve significant improvements in their operational efficiency, model reliability, and long-term sustainability, positioning themselves advantageously in an increasingly data-driven world.

6 Personal Reflection

Reflecting personally, the journey of conducting this research has significantly enriched my understanding of the intricate complexities involved in operationalizing ML models. It has reinforced the critical importance of structured methodologies such as MLOps in contemporary machine learning practices.

Throughout this research, I became increasingly aware not only of the technical challenges—such as maintaining data consistency, managing model versioning, and ensuring system scalability—but also of the indispensable organizational and cultural shifts necessary for successful implementation. Addressing these cultural dimensions included facilitating effective communication and collaboration between traditionally siloed teams such as data scientists, software engineers, and IT operations personnel.

Additionally, hands-on experience with advanced tools and technologies played a crucial role in my professional development. Specifically, becoming proficient with cutting-edge frameworks and technologies such as Kubernetes for container orchestration, Docker for containerization, GitHub Actions for CI/CD pipeline automation, and ArgoCD for continuous deployment greatly enhanced my technical capabilities. Each of these tools contributed uniquely to my ability to develop a reliable and maintainable MLOps pipeline, providing invaluable practical experience.

Looking towards the future, it is evident that the field of MLOps will continue to rapidly evolve, driven by innovations and emerging trends such as increased automation, edge computing solutions, and the adoption of responsible AI frameworks. Continued research and development will be essential to bridge existing gaps, particularly around ensuring interoperability between diverse tools, enhancing energy efficiency in computational processes, and responsibly integrating ML systems into broader technological, societal, and ethical frameworks.

Ultimately, this dissertation has made significant contributions to the academic and practical domains of MLOps. Most importantly, it has profoundly shaped my professional

perspective, equipping me with comprehensive insights and practical skills. These capabilities will enable me to proactively engage with future advancements and effectively contribute to the field of MLOps across a variety of industry sectors.

References

- Azevedo, Ana and Manuel Filipe Santos (2008). “KDD, SEMMA and CRISP-DM: a parallel overview”. In: *IADS-DM*.
- Bayram, Firas and Bestoun S Ahmed (2025). “Towards Trustworthy Machine Learning in Production: An Overview of the Robustness in MLOps Approach”. In: *ACM Computing Surveys* 57.5, pp. 1–35.
- Chapman, P et al. (2000). *CRISP-DM 1.0: Step-by-step data mining guide. The CRISP-DM consortium. SPSS*.
- Diaz-De-Arcaya, Josu et al. (2023). “A joint study of the challenges, opportunities, and roadmap of mlops and aiops: A systematic survey”. In: *ACM Computing Surveys* 56.4, pp. 1–30.
- Faubel, Leonhard and Klaus Schmid (2024). “MLOps: A Multiple Case Study in Industry 4.0”. In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, pp. 01–08.
- John, Meenu Mary, Helena Holmström Olsson, and Jan Bosch (2021). “Towards mlops: A framework and maturity model”. In: *2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, pp. 1–8.
- Kreuzberger, David, Niklas Köhl, and Michael Hirschl (2023). “Machine learning operations (MLOps): Overview, definition, and architecture”. In: *IEEE Access* 11, pp. 5353–5370.
- Mariscal, Gonzalo, Oscar Marban, and Covadonga Fernandez (2010). “A survey of data mining and knowledge discovery process models and methodologies”. In: *The knowledge engineering review* 25.2, pp. 137–166.
- Singla, Amandeep (2023). “Machine learning operations (mlops): Challenges and strategies”. In: *Journal of Knowledge Learning and Science Technology ISSN: 2959-6386 (online)* 2.3, pp. 333–340.
- Symeonidis, Georgios et al. (2022). “Mlops-definitions, tools and challenges”. In: *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, pp. 0453–0460.
- Tamburri, Damian A (2020). “Sustainable mlops: Trends and challenges”. In: *2020 22nd international symposium on symbolic and numeric algorithms for scientific computing (SYNASC)*. IEEE, pp. 17–23.
- Testi, Matteo et al. (2022). “MLOps: a taxonomy and a methodology”. In: *IEEE Access* 10, pp. 63606–63618.
- Warnett, Stephen John and Uwe Zdun (2024). “On the understandability of MLOps system architectures”. In: *IEEE Transactions on Software Engineering*.
- Wazir, Samar, Gautam Siddharth Kashyap, and Parag Saxena (2023). “Mlops: A review”. In: *arXiv preprint arXiv:2308.10908*.
- Zhou, Yue, Yue Yu, and Bo Ding (2020). “Towards mlops: A case study of ml pipeline platform”. In: *2020 International conference on artificial intelligence and computer engineering (ICAICE)*. IEEE, pp. 494–500.

A CNN Code

```
# -*- coding: utf-8 -*-
"""Dissertation-CNN.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/19
    I_hbE6LMb7bSNE1ArIJm3r4kr6dssmB
"""

# import kagglehub

# path = kagglehub.dataset_download("engmohamedsshubber/
#     violencedetectiondataset")

# print("Path to dataset files:", path)

# import shutil
# import os

# dataset_path = "/root/.cache/kagglehub/datasets/engmohamedsshubber/
#     violencedetectiondataset/versions/1"

# destination_path = "/content/drive/MyDrive/Colab Notebooks/
#     dissertation"

# if os.path.exists(destination_path):
#     shutil.rmtree(destination_path)

# shutil.copytree(dataset_path, destination_path)

# print("Dataset copied to:", destination_path)

import os
import random
import cv2
import numpy as np
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import TimeDistributed, Conv2D,
    MaxPooling2D, Flatten, LSTM, Dense, Dropout

from IPython.display import HTML
from base64 import b64encode

# To Show a Video in Notebook
def Play_Video(filepath):
    html = ''
    video = open(filepath, 'rb').read()
    src = 'data:video/mp4;base64,' + b64encode(video).decode()
```

```

        html += '<video width=640 muted controls autoplay loop><source src
        ="%s" type="video/mp4"></video>' % src
        return HTML(html)

# Classes Directories
NonViolnceVideos_Dir = "/content/drive/MyDrive/Colab Notebooks/
    dissertation/violence-detection-dataset/non-violent"
ViolnceVideos_Dir = "/content/drive/MyDrive/Colab Notebooks/
    dissertation/violence-detection-dataset/violent"

# Retrieve the list of all the video files present in the Class
    Directory.
NonViolence_files_names_list = os.listdir(NonViolnceVideos_Dir)
Violence_files_names_list = os.listdir(ViolnceVideos_Dir)

# Randomly select a video file from the Classes Directory.
Random_NonViolence_Video = random.choice(NonViolence_files_names_list)
Random_Violence_Video = random.choice(Violence_files_names_list)

# Play_Video(f"{NonViolnceVideos_Dir}/{Random_NonViolence_Video}")

# Play_Video(f"{ViolnceVideos_Dir}/{Random_Violence_Video}")

# Specify the height and width to which each video frame will be
    resized in our dataset.
IMAGE_HEIGHT , IMAGE_WIDTH = 64, 64

# Specify the number of frames of a video that will be fed to the model
    as one sequence.
SEQUENCE_LENGTH = 16

DATASET_DIR = '/content/drive/MyDrive/Colab Notebooks/dissertation/
    violence-detection-dataset'

CLASSES_LIST = ["non-violent", "violent"]

def augment_frame(frame):
    """
    Apply random augmentations to a single frame.
    Augmentations include horizontal flip, brightness adjustment,
    rotation, and adding noise.

    Parameters:
    - frame: A single video frame (numpy array) with values in [0, 1]
    or [0, 255].

    Returns:
    - Augmented frame (numpy array).
    """
    # Convert frame to uint8 for OpenCV operations (if needed)
    if frame.dtype != np.uint8:
        frame = (frame * 255).astype(np.uint8)

    # Random horizontal flip
    if random.choice([True, False]):
        frame = cv2.flip(frame, 1)

```

```

# Random brightness adjustment
if random.choice([True, False]):
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    brightness_change = random.randint(-30, 30)
    hsv[:, :, 2] = np.clip(hsv[:, :, 2] + brightness_change, 0,
255)
    frame = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

# Random rotation
if random.choice([True, False]):
    angle = random.randint(-15, 15)
    center = (frame.shape[1] // 2, frame.shape[0] // 2)
    M = cv2.getRotationMatrix2D(center, angle, 1.0)
    frame = cv2.warpAffine(frame, M, (frame.shape[1], frame.shape
[0]))

# Add random noise (keep noise in [0, 1])
if random.choice([True, False]):
    noise = np.random.normal(0, 0.05, frame.shape).astype(np.
float32)
    frame = np.clip(frame / 255.0 + noise, 0, 1) * 255.0 # Scale
back to uint8 range
    frame = frame.astype(np.uint8)

return frame

def frames_extraction_with_augmentation(video_path, augment=False):
    frames_list = []
    video_reader = cv2.VideoCapture(video_path)
    video_frames_count = int(video_reader.get(cv2.CAP_PROP_FRAME_COUNT)
)
    skip_frames_window = max(int(video_frames_count / SEQUENCE_LENGTH),
1)

    for frame_counter in range(SEQUENCE_LENGTH):
        video_reader.set(cv2.CAP_PROP_POS_FRAMES, frame_counter *
skip_frames_window)
        success, frame = video_reader.read()

        if not success:
            frame = np.zeros((IMAGE_HEIGHT, IMAGE_WIDTH, 3), dtype=np.
float32)

            resized_frame = cv2.resize(frame, (IMAGE_HEIGHT, IMAGE_WIDTH))
            normalized_frame = resized_frame / 255.0

            if augment:
                normalized_frame = augment_frame(normalized_frame)

            frames_list.append(normalized_frame)

    video_reader.release()
    return frames_list

def create_dataset_with_augmentation():
    """
    Create a dataset with data augmentation applied only to the non-
violent class.

```

```

Returns features, labels.
"""
features, labels = [], []

# Process non-violent videos (apply augmentation)
for video_file in os.listdir(NonViolnceVideos_Dir):
    video_path = os.path.join(NonViolnceVideos_Dir, video_file)

    # Original version (non-augmented)
    frames = frames_extraction_with_augmentation(video_path,
augment=False)
    if len(frames) == SEQUENCE_LENGTH:
        features.append(frames)
        labels.append(0) # Non-violent label

    # Augmented version
    frames = frames_extraction_with_augmentation(video_path,
augment=True)
    if len(frames) == SEQUENCE_LENGTH:
        features.append(frames)
        labels.append(0) # Non-violent label

# Process violent videos (no augmentation)
for video_file in os.listdir(ViolnceVideos_Dir):
    video_path = os.path.join(ViolnceVideos_Dir, video_file)
    frames = frames_extraction_with_augmentation(video_path,
augment=False)
    if len(frames) == SEQUENCE_LENGTH:
        features.append(frames)
        labels.append(1) # Violent label

    return np.array(features), np.array(labels)

# features, labels = create_dataset_with_augmentation()

# Save the dataset
# np.save("/content/drive/MyDrive/Colab Notebooks/dissertation/violence
-detection-dataset/features.npy", features)
# np.save("/content/drive/MyDrive/Colab Notebooks/dissertation/violence
-detection-dataset/labels.npy", labels)

features = np.load("/content/drive/MyDrive/Colab Notebooks/dissertation
/violence-detection-dataset/features.npy")
labels = np.load("/content/drive/MyDrive/Colab Notebooks/dissertation/
violence-detection-dataset/labels.npy")

print("Features shape:", features.shape) # Expected: (num_samples,
SEQUENCE_LENGTH, IMAGE_HEIGHT, IMAGE_WIDTH, 3)
print("Labels shape:", labels.shape) # Expected: (num_samples,)

# Check class distribution
unique, counts = np.unique(labels, return_counts=True)
print("Class distribution:", dict(zip(unique, counts)))

# Encode labels into categorical format
labels = to_categorical(labels, num_classes=len(CLASSES_LIST))

# Split dataset into training and testing sets

```

```

X_train, X_test, y_train, y_test = train_test_split(features, labels,
                                                    test_size=0.2, random_state=42)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.layers import (
    Input,
    TimeDistributed,
    Conv2D,
    MaxPooling2D,
    Flatten,
    LSTM,
    Dense,
    GlobalAveragePooling1D
)

# Define a CNN for spatial feature extraction
def build_cnn(input_shape):
    """
    Custom CNN for spatial feature extraction.
    """
    model = Sequential([
        Conv2D(64, (3, 3), activation='relu', padding='same',
input_shape=input_shape),
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(128, (3, 3), activation='relu', padding='same'),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(256, (3, 3), activation='relu', padding='same'),
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        Conv2D(512, (3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),

        Flatten()
    ])
    return model

# Parameters
SEQUENCE_LENGTH = 16 # Number of frames per video
IMAGE_HEIGHT, IMAGE_WIDTH = 64, 64 # Dimensions of each frame

```

```

NUM_CLASSES = 2 # Number of output classes (Violent and Non-Violent)

# Build the CNN
cnn = build_cnn((IMAGE_HEIGHT, IMAGE_WIDTH, 3))

# Input for video sequences
video_input = Input(shape=(SEQUENCE_LENGTH, IMAGE_HEIGHT, IMAGE_WIDTH,
3))

# Apply CNN to each frame using TimeDistributed
cnn_features = TimeDistributed(cnn)(video_input)

# LSTM for temporal feature extraction
lstm_out = LSTM(40, return_sequences=True)(cnn_features)

# TimeDistributed Dense layer for frame-level features
time_dense_out = TimeDistributed(Dense(160, activation='relu'))(
lstm_out)

# Global average pooling to summarize across frames
global_pooling = GlobalAveragePooling1D()(time_dense_out)

# Final classification layer
output = Dense(NUM_CLASSES, activation='softmax', name="last")(
global_pooling)

# Define the complete model
model = Model(inputs=video_input, outputs=output)

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# Display the model summary
model.summary()

from tensorflow.keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)
history = model.fit(
X_train, y_train,
validation_data=(X_test, y_test),
epochs=10,
batch_size=16,
callbacks=[early_stopping]
)

import matplotlib.pyplot as plt

# Extract metrics
training_loss = history.history['loss']
validation_loss = history.history['val_loss']
training_accuracy = history.history['accuracy']
validation_accuracy = history.history['val_accuracy']

# Plot Loss
plt.figure(figsize=(12, 6))

```

```

plt.plot(training_loss, label='Training Loss')
plt.plot(validation_loss, label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Plot Accuracy
plt.figure(figsize=(12, 6))
plt.plot(training_accuracy, label='Training Accuracy')
plt.plot(validation_accuracy, label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# Evaluate the model on the test set
test_loss, test_accuracy = model.evaluate(X_test, y_test, batch_size
    =16)
print(f"Test Loss: {test_loss}")
print(f"Test Accuracy: {test_accuracy}")

# Get predictions for the test set
y_pred = model.predict(X_test)

# Convert predicted probabilities to class labels
y_pred_labels = np.argmax(y_pred, axis=1)

# Convert true labels to class labels (if in one-hot encoded format)
y_true_labels = np.argmax(y_test, axis=1)

from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Classification report
print(classification_report(y_true_labels, y_pred_labels, target_names=
    CLASSES_LIST))

# Confusion matrix
conf_matrix = confusion_matrix(y_true_labels, y_pred_labels)
sns.heatmap(conf_matrix, annot=True, fmt='d', xticklabels=CLASSES_LIST,
    yticklabels=CLASSES_LIST, cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()

import numpy as np

def test_single_video(video_path, model, sequence_length, image_height,
    image_width):
    """
    Test a single video using the trained model.

    Parameters:

```

```

- video_path: Path to the video file.
- model: Trained model.
- sequence_length: Number of frames to extract for testing.
- image_height: Height to resize the video frames.
- image_width: Width to resize the video frames.

Returns:
- Predicted class and probabilities for the input video.
"""
def extract_frames_for_testing(video_path, sequence_length,
image_height, image_width):
    """
    Extracts a fixed number of frames from the video for testing.
    """
    frames_list = []
    video_reader = cv2.VideoCapture(video_path)
    video_frames_count = int(video_reader.get(cv2.
CAP_PROP_FRAME_COUNT))
    skip_frames_window = max(int(video_frames_count /
sequence_length), 1)

    for frame_counter in range(sequence_length):
        video_reader.set(cv2.CAP_PROP_POS_FRAMES, frame_counter *
skip_frames_window)
        success, frame = video_reader.read()
        if not success:
            break

        resized_frame = cv2.resize(frame, (image_width,
image_height))
        normalized_frame = resized_frame / 255.0 # Normalize to
[0, 1]
        frames_list.append(normalized_frame)

    video_reader.release()
    return np.array(frames_list)

# Extract frames from the video
frames = extract_frames_for_testing(video_path, sequence_length,
image_height, image_width)

if frames.shape[0] < sequence_length:
    print("Not enough frames in the video to meet the sequence
length requirement.")
    return

# Expand dimensions to match the input shape of the model
input_data = np.expand_dims(frames, axis=0) # Shape: (1,
sequence_length, image_height, image_width, 3)

# Make predictions
prediction = model.predict(input_data)
predicted_class = np.argmax(prediction)
prediction_probabilities = prediction[0]

print(f"Predicted Class: {CLASSES_LIST[predicted_class]}")
print(f"Prediction Probabilities: {prediction_probabilities}")

```

```

        return CLASSES_LIST[predicted_class], prediction_probabilities

video_path = "/content/drive/MyDrive/Colab Notebooks/dissertation/
violence-detection-dataset/non-violent/1 (2).mp4"
predicted_class, probabilities = test_single_video(
    video_path=video_path,
    model=model,
    sequence_length=16,
    image_height=64,
    image_width=64
)

print(f"Predicted Class: {predicted_class}")
print(f"Prediction Probabilities: {probabilities}")

```

B Transformers Code

```

1      # -*- coding: utf-8 -*-
2      """Dissertation-Transformers.ipynb
3
4      Automatically generated by Colab.
5
6      Original file is located at
7      https://colab.research.google.com/drive/1RASN-RdJgiSGN5J629dhaY--
      JIFdhJrt
8      """
9
10     # import kagglehub
11
12
13     # path = kagglehub.dataset_download("engmohamedsshubber/
      violencedetectiondataset")
14
15     # print("Path to dataset files:", path)
16
17     # import shutil
18     # import os
19
20
21     # dataset_path = "/root/.cache/kagglehub/datasets/engmohamedsshubber/
      violencedetectiondataset/versions/1"
22
23
24     # destination_path = "/content/drive/MyDrive/Colab Notebooks/
      dissertation"
25
26
27     # if os.path.exists(destination_path):
28     #     shutil.rmtree(destination_path)
29
30     # shutil.copytree(dataset_path, destination_path)
31
32     # print("Dataset copied to:", destination_path)
33
34     import os

```

```

35 import random
36 import cv2
37 import numpy as np
38 import torch
39 from tensorflow.keras.utils import to_categorical
40 from sklearn.model_selection import train_test_split
41 from tensorflow.keras.models import Sequential
42 from tensorflow.keras.layers import TimeDistributed, Conv2D,
    MaxPooling2D, Flatten, LSTM, Dense, Dropout
43
44 from IPython.display import HTML
45 from base64 import b64encode
46
47 # To Show a Video in Notebook
48 def Play_Video(filepath):
49     html = ''
50     video = open(filepath, 'rb').read()
51     src = 'data:video/mp4;base64,' + b64encode(video).decode()
52     html += '<video width=640 muted controls autoplay loop><source src
    ="%s" type="video/mp4"></video>' % src
53     return HTML(html)
54
55 # Classes Directories
56 NonViolnceVideos_Dir = "/content/drive/MyDrive/Colab Notebooks/
    dissertation/violence-detection-dataset/non-violent"
57 ViolnceVideos_Dir = "/content/drive/MyDrive/Colab Notebooks/
    dissertation/violence-detection-dataset/violent"
58
59 # Retrieve the list of all the video files present in the Class
    Directory.
60 NonViolence_files_names_list = os.listdir(NonViolnceVideos_Dir)
61 Violence_files_names_list = os.listdir(ViolnceVideos_Dir)
62
63 # Randomly select a video file from the Classes Directory.
64 Random_NonViolence_Video = random.choice(NonViolence_files_names_list)
65 Random_Violence_Video = random.choice(Violence_files_names_list)
66
67 # Play_Video(f"{NonViolnceVideos_Dir}/{Random_NonViolence_Video}")
68
69 # Play_Video(f"{ViolnceVideos_Dir}/{Random_Violence_Video}")
70
71 # Specify the height and width to which each video frame will be
    resized in our dataset.
72 IMAGE_HEIGHT , IMAGE_WIDTH = 64, 64
73
74 # Specify the number of frames of a video that will be fed to the model
    as one sequence.
75 SEQUENCE_LENGTH = 16
76
77
78 DATASET_DIR = '/content/drive/MyDrive/Colab Notebooks/dissertation/
    violence-detection-dataset'
79
80 CLASSES_LIST = ["non-violent", "violent"]
81
82 def augment_frame(frame):
83     """
84     Apply random augmentations to a single frame.

```

```

85     Augmentations include horizontal flip, brightness adjustment,
      rotation, and adding noise.
86
87     Parameters:
88     - frame: A single video frame (numpy array) with values in [0, 1]
      or [0, 255].
89
90     Returns:
91     - Augmented frame (numpy array).
92     """
93     # Convert frame to uint8 for OpenCV operations (if needed)
94     if frame.dtype != np.uint8:
95         frame = (frame * 255).astype(np.uint8)
96
97     # Random horizontal flip
98     if random.choice([True, False]):
99         frame = cv2.flip(frame, 1)
100
101     # Random brightness adjustment
102     if random.choice([True, False]):
103         hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
104         brightness_change = random.randint(-30, 30)
105         hsv[:, :, 2] = np.clip(hsv[:, :, 2] + brightness_change, 0,
255)
106         frame = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
107
108     # Random rotation
109     if random.choice([True, False]):
110         angle = random.randint(-15, 15)
111         center = (frame.shape[1] // 2, frame.shape[0] // 2)
112         M = cv2.getRotationMatrix2D(center, angle, 1.0)
113         frame = cv2.warpAffine(frame, M, (frame.shape[1], frame.shape
[0]))
114
115     # Add random noise (keep noise in [0, 1])
116     if random.choice([True, False]):
117         noise = np.random.normal(0, 0.05, frame.shape).astype(np.
float32)
118         frame = np.clip(frame / 255.0 + noise, 0, 1) * 255.0 # Scale
back to uint8 range
119         frame = frame.astype(np.uint8)
120
121     return frame
122
123 def frames_extraction_with_augmentation(video_path, augment=False):
124     frames_list = []
125     video_reader = cv2.VideoCapture(video_path)
126     video_frames_count = int(video_reader.get(cv2.CAP_PROP_FRAME_COUNT)
)
127     skip_frames_window = max(int(video_frames_count / SEQUENCE_LENGTH),
1)
128
129     for frame_counter in range(SEQUENCE_LENGTH):
130         video_reader.set(cv2.CAP_PROP_POS_FRAMES, frame_counter *
skip_frames_window)
131         success, frame = video_reader.read()
132
133         if not success:

```

```

134         frame = np.zeros((IMAGE_HEIGHT, IMAGE_WIDTH, 3), dtype=np.
float32)
135
136         resized_frame = cv2.resize(frame, (IMAGE_HEIGHT, IMAGE_WIDTH))
137         normalized_frame = resized_frame / 255.0
138
139         if augment:
140             normalized_frame = augment_frame(normalized_frame)
141
142         frames_list.append(normalized_frame)
143
144     video_reader.release()
145     return frames_list
146
147 def create_dataset_with_augmentation():
148     """
149     Create a dataset with data augmentation applied only to the non-
violent class.
150     Returns features, labels.
151     """
152     features, labels = [], []
153
154     # Process non-violent videos (apply augmentation)
155     for video_file in os.listdir(NonViolnceVideos_Dir):
156         video_path = os.path.join(NonViolnceVideos_Dir, video_file)
157
158         # Original version (non-augmented)
159         frames = frames_extraction_with_augmentation(video_path,
augment=False)
160         if len(frames) == SEQUENCE_LENGTH:
161             features.append(frames)
162             labels.append(0) # Non-violent label
163
164         # Augmented version
165         frames = frames_extraction_with_augmentation(video_path,
augment=True)
166         if len(frames) == SEQUENCE_LENGTH:
167             features.append(frames)
168             labels.append(0) # Non-violent label
169
170     # Process violent videos (no augmentation)
171     for video_file in os.listdir(ViolnceVideos_Dir):
172         video_path = os.path.join(ViolnceVideos_Dir, video_file)
173         frames = frames_extraction_with_augmentation(video_path,
augment=False)
174         if len(frames) == SEQUENCE_LENGTH:
175             features.append(frames)
176             labels.append(1) # Violent label
177
178     return np.array(features), np.array(labels)
179
180 # features, labels = create_dataset_with_augmentation()
181
182 # Save the dataset
183 # np.save("/content/drive/MyDrive/Colab Notebooks/dissertation/violence
-detection-dataset/features.npy", features)
184 # np.save("/content/drive/MyDrive/Colab Notebooks/dissertation/violence
-detection-dataset/labels.npy", labels)

```

```

185
186 features = np.load("/content/drive/MyDrive/Colab Notebooks/dissertation
    /violence-detection-dataset/features.npy")
187 labels = np.load("/content/drive/MyDrive/Colab Notebooks/dissertation/
    violence-detection-dataset/labels.npy")
188
189 print("Features shape:", features.shape) # Expected: (num_samples,
    SEQUENCE_LENGTH, IMAGE_HEIGHT, IMAGE_WIDTH, 3)
190 print("Labels shape:", labels.shape) # Expected: (num_samples,)
191
192 import torch
193 from sklearn.model_selection import train_test_split
194
195 # Convert to PyTorch tensor and rearrange axes to [N, T, C, H, W]
196 features = torch.tensor(features).permute(0, 1, 4, 2, 3).float() #
    from [N, T, H, W, C]
197
198 # Encode labels if they are categorical
199 if labels.ndim > 1:
200     labels = torch.tensor(labels.argmax(axis=1)).long()
201 else:
202     labels = torch.tensor(labels).long()
203
204 # Train-test split
205 X_train, X_test, y_train, y_test = train_test_split(features, labels,
    test_size=0.2, random_state=42)
206
207 # Check class distribution
208 unique, counts = np.unique(labels, return_counts=True)
209 print("Class distribution:", dict(zip(unique, counts)))
210
211 from torch.utils.data import Dataset, DataLoader
212
213 class VideoDataset(Dataset):
214     def __init__(self, videos, labels):
215         self.videos = videos
216         self.labels = labels
217
218     def __len__(self):
219         return len(self.labels)
220
221     def __getitem__(self, idx):
222         return self.videos[idx], self.labels[idx]
223
224 train_dataset = VideoDataset(X_train, y_train)
225 test_dataset = VideoDataset(X_test, y_test)
226
227 train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
228 test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)
229
230 !pip install timesformer-pytorch
231
232 from timesformer_pytorch import TimeSformer
233
234 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
235
236 model = TimeSformer(
237     image_size=64,

```

```

238     num_frames=X_train.shape[1],
239     num_classes=len(torch.unique(y_train)),
240     dim=512,
241     depth=4,
242     heads=8,
243     patch_size=16,
244     attn_dropout=0.1,
245     ff_dropout=0.1
246 ).to(device)
247
248 import torch.nn as nn
249 import torch.optim as optim
250
251 criterion = nn.CrossEntropyLoss()
252 optimizer = optim.Adam(model.parameters(), lr=1e-4)
253
254 for epoch in range(10):
255     model.train()
256     total_loss = 0
257     for videos, labels in train_loader:
258         videos, labels = videos.to(device), labels.to(device)
259         optimizer.zero_grad()
260         outputs = model(videos)
261         loss = criterion(outputs, labels)
262         loss.backward()
263         optimizer.step()
264         total_loss += loss.item()
265     print(f"Epoch {epoch+1} | Loss: {total_loss:.4f}")
266
267 model.eval()
268 correct, total = 0, 0
269 with torch.no_grad():
270     for videos, labels in test_loader:
271         videos, labels = videos.to(device), labels.to(device)
272         outputs = model(videos)
273         _, preds = torch.max(outputs, 1)
274         correct += (preds == labels).sum().item()
275         total += labels.size(0)
276
277 print(f"Test Accuracy: {100 * correct / total:.2f}%")
278
279 from sklearn.metrics import classification_report, confusion_matrix
280 import seaborn as sns
281 import matplotlib.pyplot as plt
282
283 # Evaluation
284 model.eval()
285 all_preds = []
286 all_labels = []
287
288 with torch.no_grad():
289     for batch_x, batch_y in test_loader:
290         batch_x, batch_y = batch_x.to(device), batch_y.to(device)
291         outputs = model(batch_x)
292         _, preds = torch.max(outputs, 1)
293         all_preds.extend(preds.cpu().numpy())
294         all_labels.extend(batch_y.cpu().numpy())
295

```

```

296 # Classification Report
297 print("\nClassification Report:\n")
298 print(classification_report(all_labels, all_preds))
299
300 # Confusion Matrix
301 cm = confusion_matrix(all_labels, all_preds)
302 plt.figure(figsize=(8,6))
303 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
304 plt.title("Confusion Matrix")
305 plt.xlabel("Predicted")
306 plt.ylabel("Actual")
307 plt.show()
308
309 import random
310
311 model.eval()
312 idxs = random.sample(range(len(X_test)), 5)
313
314 for idx in idxs:
315     video = X_test[idx].unsqueeze(0).to(device)
316     label = y_test[idx].item()
317
318     with torch.no_grad():
319         output = model(video)
320         pred = torch.argmax(output, dim=1).item()
321
322     print(f"\nSample Index: {idx}")
323     print(f"Actual Label: {label} | Predicted Label: {pred}")
324
325     # Visualize first frame (if you want to show the raw image)
326     frame = video[0, 0].cpu().numpy().transpose(1, 2, 0)
327     frame = (frame - frame.min()) / (frame.max() - frame.min()) #
    normalize to [0, 1]
328     plt.imshow(frame)
329     plt.title(f"True: {label}, Pred: {pred}")
330     plt.axis('off')
331     plt.show()
332
333 # Save the trained model
334 torch.save(model.state_dict(), '/content/drive/MyDrive/Colab Notebooks/
    dissertation/transformer_model.pth')
335

```

C App Code

```

1     import streamlit as st
2     import numpy as np
3     import cv2
4     import os
5     import torch
6     import torch.nn as nn
7     from timesformer_pytorch import TimeSformer
8
9     # Constants
10    SEQUENCE_LENGTH = 16
11    IMAGE_HEIGHT, IMAGE_WIDTH = 64, 64

```

```

12 CLASSES_LIST = ["Non-Violent", "Violent"]
13
14 # Define the Transformer model class (must match training definition)
15 class TransformerClassifier(nn.Module):
16     def __init__(self, input_dim=3, num_classes=2, nhead=4, num_layers
        =2):
17         super().__init__()
18         self.embedding = nn.Linear(input_dim, 128)
19         encoder_layer = nn.TransformerEncoderLayer(d_model=128, nhead=
        nhead)
20         self.transformer = nn.TransformerEncoder(encoder_layer,
        num_layers=num_layers)
21         self.classifier = nn.Linear(128, num_classes)
22
23     def forward(self, x):
24         x = self.embedding(x) # [B, T, 128]
25         x = self.transformer(x) # [B, T, 128]
26         x = x.mean(dim=1) # Global average pool
27         return self.classifier(x)
28
29
30
31 @st.cache_resource
32 def load_trained_model():
33     model = TimeSformer(
34         image_size=64,
35         num_frames=SEQUENCE_LENGTH,
36         num_classes=len(CLASSES_LIST),
37         dim=512,
38         depth=4,
39         heads=8,
40         patch_size=16,
41         attn_dropout=0.1,
42         ff_dropout=0.1
43     )
44     model.load_state_dict(torch.load("transformer_model.pth",
        map_location=torch.device('cpu')))
45     model.eval()
46     return model
47
48 # Frame extraction
49 def extract_frames(video_path, sequence_length, image_height,
        image_width):
50     frames_list = []
51     video_reader = cv2.VideoCapture(video_path)
52     video_frames_count = int(video_reader.get(cv2.CAP_PROP_FRAME_COUNT)
        )
53     skip_frames_window = max(int(video_frames_count / sequence_length),
        1)
54
55     for frame_counter in range(sequence_length):
56         video_reader.set(cv2.CAP_PROP_POS_FRAMES, frame_counter *
        skip_frames_window)
57         success, frame = video_reader.read()
58         if not success:
59             break
60         resized_frame = cv2.resize(frame, (image_width, image_height))
61         normalized_frame = resized_frame / 255.0

```

```

62         frames_list.append(normalized_frame)
63
64     video_reader.release()
65     return np.array(frames_list)
66
67 # PyTorch inference
68 def predict_video_class(video_path, model):
69     frames = extract_frames(video_path, SEQUENCE_LENGTH, IMAGE_HEIGHT,
70 IMAGE_WIDTH)
71     if len(frames) < SEQUENCE_LENGTH:
72         st.error("The video is too short. Please upload a longer video.
73 ")
74         return None, None
75
76     frames = torch.tensor(frames).float() # [T, H,
77 W, C]
78     frames = frames.permute(0, 3, 1, 2) # [T, C,
79 H, W]
80     input_tensor = frames.unsqueeze(0) # [1, T,
81 C, H, W]
82
83     with torch.no_grad():
84         output = model(input_tensor)
85         probs = torch.softmax(output, dim=1).cpu().numpy()[0]
86         pred_idx = np.argmax(probs)
87     return CLASSES_LIST[pred_idx], probs
88
89 # Streamlit App
90 def main():
91     st.title("Violence Detection (Transformer Model)")
92     st.write("Upload a video to classify it as **Non-Violent** or **
93 Violent**.")
94
95     video_file = st.file_uploader("Upload a Video", type=["mp4", "avi",
96 "mov", "mkv"])
97
98     if video_file is not None:
99         temp_video_path = "temp_video.mp4"
100         with open(temp_video_path, "wb") as f:
101             f.write(video_file.read())
102
103         st.video(temp_video_path)
104
105         model = load_trained_model()
106
107         with st.spinner("Analyzing the video..."):
108             predicted_class, probabilities = predict_video_class(
109 temp_video_path, model)
110
111         if predicted_class is not None:
112             st.success(f"Prediction: **{predicted_class}**")
113             st.write("Confidence Scores:")
114             for i, class_name in enumerate(CLASSES_LIST):
115                 st.write(f"{class_name}: {probabilities[i]:.2f}")
116
117         if os.path.exists(temp_video_path):
118             os.remove(temp_video_path)

```

```

112 if __name__ == "__main__":
113     main()
114

```

D Dockerfile Code

```

    # Dockerfile for Streamlit app with OpenCV
FROM python:3.10-slim

# Set working directory
WORKDIR /app

# Install system dependencies needed for OpenCV and Streamlit
RUN apt-get update && \
    apt-get install -y \
        ffmpeg \
        libsm6 \
        libxext6 \
        libgl1 \
        libglib2.0-0 && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

# Copy and install Python packages
COPY requirements.txt .
RUN pip install --upgrade pip && pip install --no-cache-dir -r requirements.txt

# Copy app files
COPY . .

# Expose Streamlit's default port
EXPOSE 8501

# Start Streamlit app
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0",

```

E GitHub Actions Code

```

    name: Build and Push Docker Image

on:
  push:
    branches: [ main ]
    paths:
      - 'app.py'
      - 'requirements.txt'
      - 'Dockerfile'

```

```

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to DockerHub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }}
          password: ${ secrets.DOCKER_PASSWORD }}

      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: alihraza/violence-detector:latest
    \end{lstlisting}
\end{center}
\section{deployment.yaml Code}
\begin{center}
  \begin{lstlisting}
    apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: violence-detector
  spec:
    replicas: 1
    selector:
      matchLabels:
        app: violence-detector
    template:
      metadata:
        labels:
          app: violence-detector
      spec:
        containers:
          - name: violence-detector
            image: alihraza/violence-detector:latest
            ports:

```

```

    - containerPort: 8501
  resources:
    requests:
      memory: "128Mi"
      cpu: "250m"
    limits:
      memory: "256Mi"
      cpu: "500m"
---

apiVersion: v1
kind: Service
metadata:
  name: violence-detector-service
spec:
  type: NodePort
  selector:
    app: violence-detector
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8501

```

F application.yaml(ArgoCD) Code

```

  apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: violence-detector
  namespace: argocd
  annotations:
    argocd-image-updater.argoproj.io/image-list: violence-detector=alihraza/violence-
    argocd-image-updater.argoproj.io/violence-detector.update-strategy: latest
spec:
  project: default
  source:
    repoURL: https://github.com/alihraza099/dissertation_k8s.git
    targetRevision: main
    path: .
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```